Republic of Tunisia
Ministry of High Education and Scientific Research
***
University of Monastir
High Institute of Computer Science and Mathematics of Monastir
Department of Technology
***

Graduation Project

In order to obtain the
National Diploma of Engineering in Applied Sciences and Technology

Major:
Electronics: Microelectronics

Presented by:

Najd Elaoud and Selim Triki

Design and development of the IOBOX-BWS industrial platform for data acquisition, processing, and control

---

Presented *X* **July** 2026 **in front of the jury members :**

Mrs. UNKOWN	President
Mr. UNKOWN	Lecturer
Mr. Abdessalem Ben Abdelali	Academic Supervisor
Mr. Jamel Hmidi	Industrial Supervisor

Academic year: 2025/2026

ACKNOWLEDGEMENTS

I am humbled and grateful to all who have helped me translate these ideas into something concrete that goes far beyond a simple level.

I would like to express my sincere gratitude to my academic supervisor, **Mr. Abdessalem BEN ABDELALI**, for his guidance throughout this project, for reviewing my work, and for his valuable remarks and suggestions that helped improve the quality of this report.

I would also like to express my deepest appreciation to my industrial supervisor, **Mr. Jamel HMIDI**, for his continuous support, technical guidance, and availability throughout the internship. His expertise, advice, and encouragement were invaluable to the successful completion of this project.

Last but not least, I am deeply grateful to all the jury members **Mr. UNKOWN** and **Mrs. UNKOWN** for agreeing to evaluate this work.

Without your support, this accomplishment would not have been achieved. I really appreciate it.

DEDICATION

To my precious family, I would never be where I am now without you.

To my dear father ***Noureddine****:*
Thank you for your hard work, guidance, and the values you have instilled in me throughout my life. Your sacrifices and dedication have always inspired me to strive for success. May God bless you with health, happiness, and a long life.

To my dear mother ***Soufia****:*
You have done more than a mother can do to ensure that her children follow the right path in their lives and studies. I dedicate this work to you as a testimony of my deep love. May God, the Almighty, preserve you and grant you health, long life, and happiness.

To my dear sister ***Nibras****:*
Thank you for your constant support, encouragement, and affection. Having you by my side has always been a source of strength and motivation.

To my dear Friends, Words can't express how grateful I am to have your friendship in my life. Thank you for supporting me, accepting me, and being wonderful friends.

Lastly, to all the kind people I met during my six years here, to all the institute students, professors, administrators and workers, thank you for being part of my exciting university career.

Najd Elaoud

DEDICATION

*"What you leave behind is not what is engraved in stone monuments, but what is woven into the lives of others." — **Pericles***

*To my dear father **Lotfi**:*
This project and my success in my academic journey could not have been achieved without your presence in my life. I am deeply grateful for the comfort and opportunities you provided for us, and for teaching me that with dedication and perseverance nothing is impossible. I hope that one day I can repay you, even in a small way, for all that you have done.

*To my dear mother **Monia**:*
The love you gave us made us stronger, and the dedication and support you offered whenever we fell down or felt low is the most precious gift in life. Thank you for being the light that guided us and for showing us the strength of unconditional love.

*To my dear sister **Ahlem**:*
Thank you for bringing happiness into my life and for being my partner through both struggles and hard work. Your emotional support has been invaluable, and I am truly grateful for the strength and companionship you have given me.

*To my dear brother **Youssef**:*
I cannot wait to see you thrive and shine in life, overcoming every turbulence along the way. I will always be here as your mentor whenever you need guidance.

*To my dear **friends** ,Thank you for all the support you have given me and for the joy you bring into my life. I am truly grateful that you accept every side of my personality, and I cherish the real human connection we share. Seeing you succeed in life is part of my own dreams, and I look forward to celebrating your achievements alongside you.*

Selim Triki

ABSTRACT

The development of flexible and interoperable industrial platforms is essential for addressing the growing integration challenges of Industrial Internet of Things (IIoT) systems. This report presents the design and development of the IOBOX, a configurable industrial Input/Output platform intended to simplify the acquisition, processing, control, and exchange of data between sensors, actuators, and supervisory systems.

The developed platform is based on an STM32 microcontroller and integrates multiple industrial communication interfaces, peripheral drivers, and sensor management modules within a modular software architecture. The implementation emphasizes hardware abstraction, software reusability, and scalability, enabling efficient integration of heterogeneous devices while reducing development effort.

The resulting solution provides a versatile and reusable platform for industrial monitoring, automation, and IoT applications. By improving interoperability and simplifying system integration, the IOBOX contributes to the development of more efficient and scalable industrial solutions.

Keywords: Industrial IoT, Embedded Systems, STM32, IO-Box, Data Acquisition, Industrial Communication Protocols, Automation, Software Architecture.

RESUME

Le développement de plateformes industrielles flexibles et interopérables est devenu essentiel pour répondre aux défis croissants d'intégration des systèmes de l'Internet Industriel des Objets (IIoT). Ce rapport présente la conception et le développement de l'IOBOX, une plateforme industrielle d'Entrées/Sorties configurable destinée à simplifier l'acquisition, le traitement, le contrôle et l'échange de données entre les capteurs, les actionneurs et les systèmes de supervision.

La plateforme développée repose sur un microcontrôleur STM32 et intègre plusieurs interfaces de communication industrielle, pilotes de périphériques et modules de gestion de capteurs au sein d'une architecture logicielle modulaire. L'implémentation met l'accent sur l'abstraction matérielle, la réutilisabilité logicielle et l'évolutivité, permettant une intégration efficace de dispositifs hétérogènes tout en réduisant les efforts de développement.

La solution obtenue constitue une plateforme polyvalente et réutilisable pour les applications de supervision industrielle, d'automatisation et d'IoT. En améliorant l'interopérabilité et en simplifiant l'intégration des systèmes, l'IOBOX contribue au développement de solutions industrielles plus efficaces et évolutives.

Mots-clés : Internet Industriel des Objets (IIoT), Systèmes Embarqués, STM32, IOBOX, Acquisition de Données, Protocoles de Communication Industrielle, Automatisation.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

GLOSSARY OF ACRONYMS

BSP: Board Support Package

UART : Universal Asynchronous Receiver-Transmitter

LIN: Local Interconnect Network

USB : Universal Serial Bus

CAN : Controller Area Network

I2C : Inter-Integrated Circuit

SPI : Serial Peripheral Interface

DAC : Digital to Analog Converter

ADC : Analog to Digital Converter

PWM : Pulse Width Modulation

DMA : Direct Memory Access

GPIO : General Purpose Input Output

PID : Proportional Integral Derivative

GENERAL INTRODUCTION

Industrial systems are increasingly relying on connected devices capable of acquiring, processing, and exchanging information in real time. In such environments, sensors, actuators, and supervisory systems must interact efficiently through various communication interfaces and protocols. However, integrating these heterogeneous components often requires significant development effort, particularly when flexibility, scalability, and interoperability are required.

To address these challenges, Be Wireless Solutions (BWS) launched the development of the IOBox platform, a configurable industrial Input/Output system designed to serve as a universal interface between field devices and monitoring or control systems. The platform is intended to simplify the integration of sensors and actuators while providing data acquisition, signal processing, control, and communication functionalities within a single solution.

The IOBox supports multiple industrial communication protocols and offers both analog and digital input/output capabilities. Its modular architecture enables adaptation to various industrial applications, ranging from environmental monitoring and energy management to process automation and equipment supervision. By centralizing these functionalities in a single platform, the IOBox reduces deployment complexity and accelerates the development of industrial IoT solutions.

The objective of this end-of-study project is to participate in the design and implementation of the IOBox platform. The work focuses on the development of embedded software components, communication interfaces, peripheral drivers, sensor integration, and system architecture required to ensure reliable operation in industrial environments.

This report, which crowns the work entrusted to us, is structured around three chapters:

Chapter 1 presents the general context of the project, the host company, the problem statement, the project objectives, and the adopted development methodology.

Chapter 2 introduces the general concept of the IOBox platform and describes the hardware components, software tools, communication protocols, sensors, and peripherals used throughout the project.

Chapter 3 details the design and implementation of the platform, including the software architecture, peripheral configuration, driver development, communication management, sensor integration, and system validation.

CHAPTER 1

GENERAL PROJECT CONTEXT

1.1 Introduction

This chapter gives an overview of the general setting for the graduation project and the environment where it took place. It starts by introducing the host company, Be Wireless Solutions (BWS), explaining its structure, the kinds of work it does, and the main IoT solutions it offers. The chapter then outlines the project's requirements by explaining the problem that was identified, the idea behind the IO-Box platform, the development method used, and the project's goals and anticipated effects. This summary helps set the stage for understanding the technical information discussed in the following chapters.

1.2 Academic Institution

The Higher Institute of Computer Science and Mathematics at the University of Monastir (ISIMM) was created by decree number 1623 on July 9, 2002. It is a public, scientific higher education institution that is supervised by the Ministry of Higher Education and Scientific Research[2]. ISIMM provides engineering programs in Computer Science, Mathematics, and Electronics. This graduation project was done in the Electronics department, with a focus on Microelectronics, as part of the requirements for the National Diploma in Engineering in Applied Sciences and Technology.

1.3 Host Company Presentation

1.3.1 Company Overview

Be Wireless Solutions (BWS) is a Tunisian engineering company that focuses on IoT systems, embedded electronics, and industrial monitoring. The company creates and makes connected devices, communication gateways, smart sensors, and cloud-based platforms that help with real-time monitoring, control, and optimization of industrial processes. BWS has knowledge in hardware design, embedded software development, wireless communication, and cloud services, allowing it to provide full industrial IoT solutions across various industries. Since it was started, BWS has kept growing its business and formed partnerships both in Tunisia and around the world. The company has become skilled in areas like industrial automation, energy management, smart farming, fleet management, environmental monitoring, and remote equipment oversight.

Figure 1.1: Be Wireless Solutions (BWS) logo

1.3.2 Organization Chart

BWS has different departments that work together during the whole process of creating industrial and IoT solutions. These departments are management, research and development, hardware design, embedded software development, cloud services, technical support, and business development.

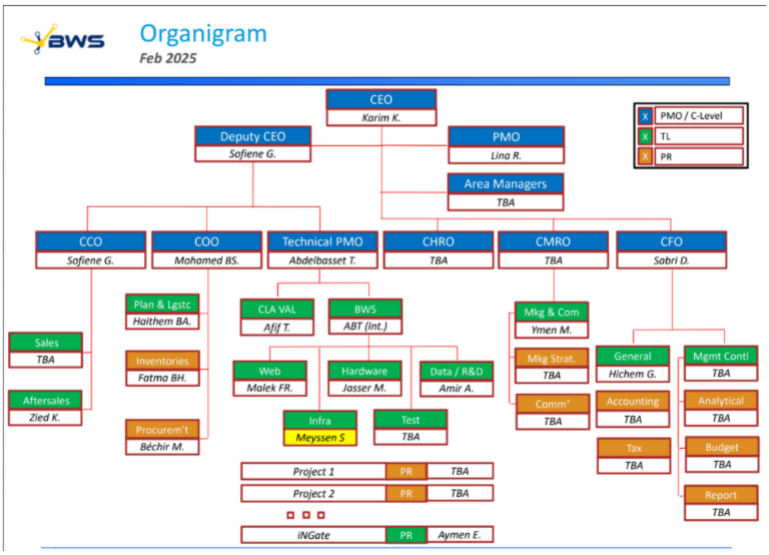

Figure 1.2: Organization chart of Be Wireless Solutions

1.3.3 Main Activities and Services

BWS creates many different industrial IoT solutions that help make operations more efficient, manage resources better, and keep track of processes more effectively.

Energy Monitoring and Management

BWS provides intelligent energy monitoring systems capable of collecting, analyzing, and visualizing electricity consumption data in real time. These solutions help organizations identify energy-intensive equipment, optimize consumption, reduce operational costs, and improve environmental performance.

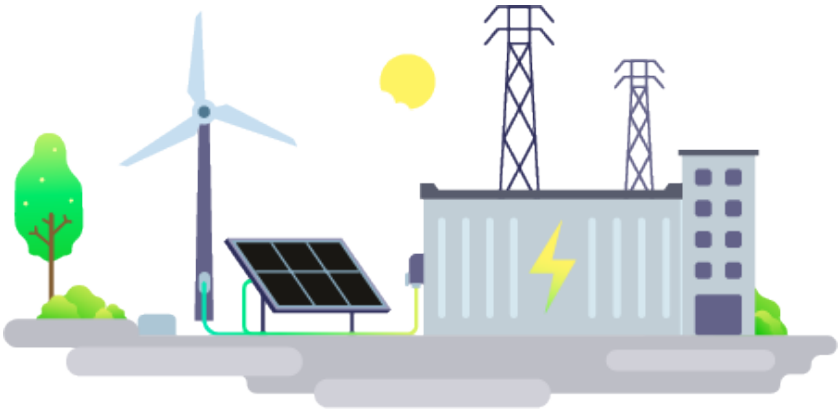

Figure 1.3: Electricity consumption monitoring solution

Water Monitoring Solutions

The company provides smart water management solutions that allow for remote reading of meters, detecting leaks, monitoring unusual water use, and helping to use water resources more efficiently.

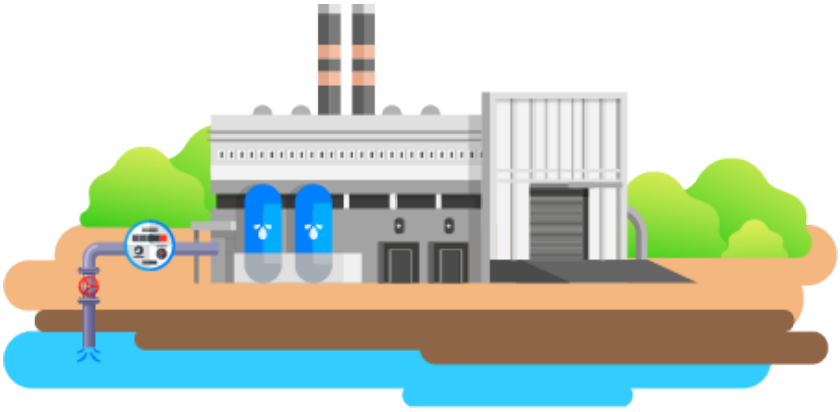

Figure 1.4: Water consumption monitoring solution

Fleet and Fuel Management

BWS develops fleet management systems that provide real-time vehicle tracking, fuel consumption analysis, route optimization, and driver behavior monitoring.

Figure 1.5: Fleet and fuel management solution

Remote Monitoring and Equipment Control

The company provides remote supervision solutions that allow operators to monitor industrial equipment, detect anomalies, generate alerts, and perform remote control operations through dedicated software platforms.

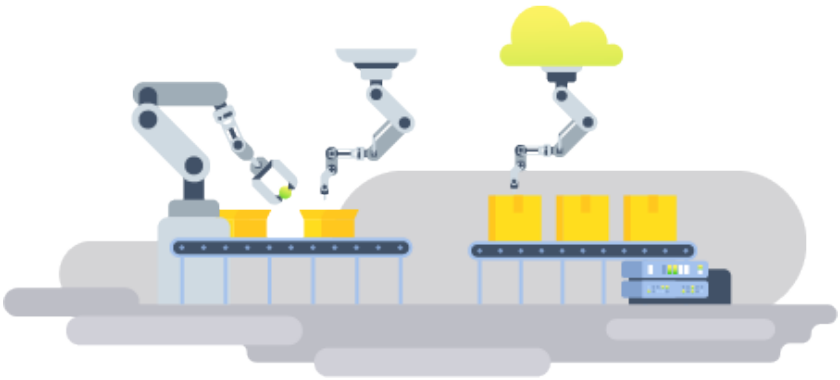

Figure 1.6: Remote equipment monitoring and control

Cold Chain Monitoring

BWS also offers cold-chain monitoring systems intended for warehouses, refrigerated transportation, and pharmaceutical storage facilities. These solutions ensure compliance with temperature requirements and provide instant alerts when abnormal conditions occur.

Figure 1.7: Cold-chain monitoring solution

1.4 Project Specification

1.4.1 Problem Statement

Industrial IoT systems usually use many sensors, actuators, and communication interfaces. Before this project, BWS created custom firmware for each new client setup. This meant they had to reconfigure and rebuild drivers every time there was a new mix of sensors, actuators, and communication interfaces. This method made development

take longer, limited the ability to reuse code between projects, and made it harder to maintain the system when hardware changed. To fix these issues, BWS started the IOBOX project. The goal was to build a flexible, hardware-independent platform that could support various setups using one common firmware base.

In industrial settings, there are often different communication protocols and interface standards. Handling these through custom solutions adds complexity and makes it harder to scale the system.

To tackle these challenges, Be Wireless Solutions began developing the IOBOX platform. The aim is to offer a customizable and reusable system for industrial data collection and control. This system aims to make adding peripherals easier while keeping the system flexible, easy to move between setups, and simple to maintain.

1.4.2 Presentation of the IOBOX Platform

The IOBOX is a flexible industrial system that helps connect sensors, actuators, and communication tools in industrial setups.

It uses a modular design that separates the parts that depend on specific hardware from the parts that handle the main functions. This is done using special software tools like drivers, a Board Support Package, and a Hardware Abstraction Layer, which make it easier to use across different systems and keep it easy to update.

The IOBOX can work with various types of industrial equipment such as analog and digital inputs and outputs, communication ports, and measurement devices. Its software setup can be changed to fit different industrial needs without needing big changes to the firmware.

By linking the hardware to the software in a simple way, the IOBOX makes it easier to put together and use in industrial enviorments.

1.4.3 Project Objectives

The main goal of this project is to create and build the IOBOX platform, which can collect, handle, control, and send industrial data as it happens. More specifically, the project aims to:

- Create and set up a flexible industrial system for gathering data and managing control.
- Provide a unified software abstraction layer for heterogeneous peripherals.
- Support configurable analog and digital input/output functionalities.

- Implement industrial communication protocols including Modbus RTU over RS485, CAN, LIN, and multi-channel UART, enabling interoperability with standard industrial equipment.
- Develop a modular software architecture based on abstraction layers and reusable software components.
- Design the software architecture to allow new sensors, actuators, and communication interfaces to be added with minimal modification to existing code, through compile-time feature flags and a consistent BSP layer.

The developed platform contributes to reducing development effort and deployment time by providing a standardized and reusable embedded architecture capable of supporting a wide range of industrial applications.

1.4.4 Development Methodology

The IOBOX platform was built using the V-Model, a structured way of developing systems that's commonly used in embedded and industrial projects.

The V-Model connects each step of the development process with a matching testing step. This helps make sure that every part of the system, from the original requirements to the final tests, can be clearly traced back to its source.

In this project, the V-Model was followed closely. During the requirements phase, the platform's features and technical needs were set based on what BWS needed. The system design phase created the four-layer software structure and picked the right hardware parts. The software design phase developed the BSP module interfaces, the communication manager designs, and the sensor driver details. The implementation phase included writing all the BSP drivers, sensor managers, communication modules, and the application layer. Testing was done by checking each BSP module individually, testing the sensor data flow and Modbus communication together, and finally testing the whole system to make sure it works as planned.

Table 1.1: Application of the V-Model to the Project

V-Model Phase	**Project Activity**
Requirements Analysis	Platform specifications derived from BWS requirements.
System Design	Hardware selection and software architecture definition.
Software Design	BSP API design and manager module interface definition.
Implementation	Development of BSP drivers, sensor managers, Modbus communication, CAN communication, and filtering algorithms.
Unit Testing	Validation of individual BSP modules and drivers.
Integration Testing	Verification of sensor chains and end-to-end Modbus communication.
System Testing	Complete platform validation on the target hardware.

The ascending branch focuses on checking and making sure everything works correctly through unit testing, integration testing, system testing, and final acceptance testing.

The use of the V-Model helps improve the quality of the software, lowers the risks during development, and makes it easier to manage the project all through its different stages.

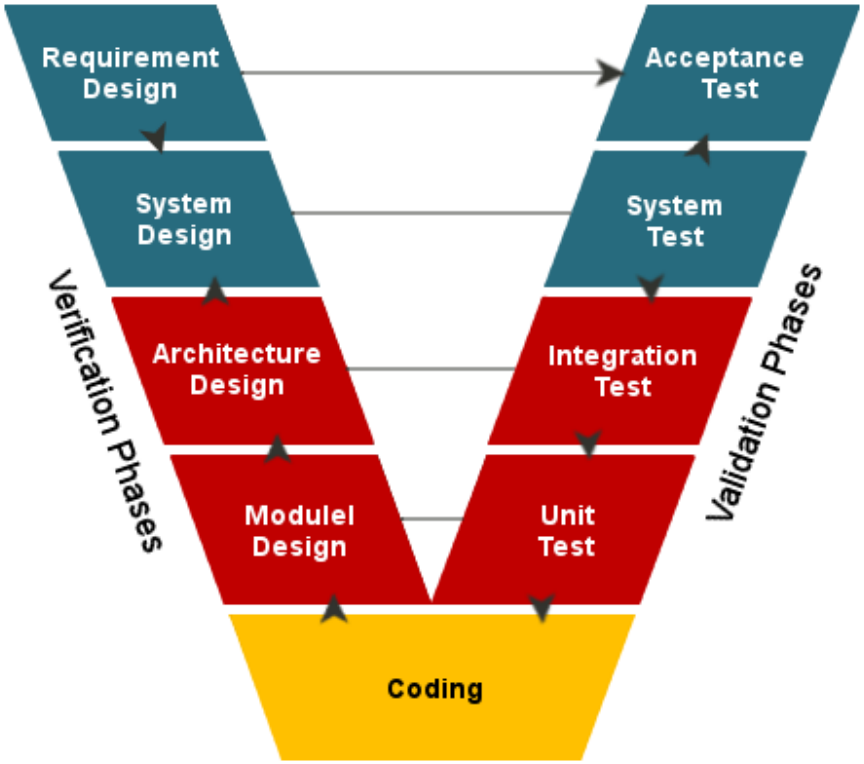

Figure 1.8: V-Model development lifecycle

1.5 Conclusion

This chapter explained the main structure of the project. It started by introducing the university and the company where the project is based. Then, it gave a summary of what the company does and the technologies they use. Next, it went into more detail about the project's requirements, the problem it aims to solve, its goals, and the approach used to develop it.

The next chapter talks about the technical setup of the IOBox platform. It covers the physical parts of the system, the software tools used for development, the ways devices communicate with each other, and the extra parts of the system that support the main functions.

CHAPTER 2

GENERAL CONCEPT AND SYSTEM COMPONENTS

2.1 Introduction

«««< HEAD This chapter talks about the technical setup and the main parts used to build the IO-Box platform. It covers the hardware and software that make up the system, like the STM32G474 microcontroller, external peripherals, sensors, tools for development, and communication protocols.

The chapter also explains the main STM32 parts used in the project, such as Analog-to-Digital Converters (ADC), Digital-to-Analog Converters (DAC), timers, and Pulse Width Modulation (PWM) features. Knowing about these technologies is important for understanding the design and how things work, which will be explained in the next chapters. ======= This chapter presents the technical environment and the main components used in the development of the IOBOX platform. It introduces the hardware and software resources that form the basis of the system, including the STM32G474 microcontroller, external peripherals, sensors, development tools, and communication protocols. »»»> a15e0a982f2f6d5766b8147a7f12c9c22426e956

«««< HEAD

2.2 General Concept of the IO-Box Platform

The IO-Box is a customizable industrial Input/Output system that helps connect sensors, actuators, and communication tools in IoT and automation setups. =======

2.3 General Concept of the IOBOX Platform

The IOBOX is a configurable industrial Input/Output platform designed to facilitate the integration of sensors, actuators, and industrial communication interfaces within IoT and automation systems. »»»> a15e0a982f2f6d5766b8147a7f12c9c22426e956

It works as a bridge between field devices and control systems by gathering data from sensors, handling that data, managing external equipment, and sharing information using different communication protocols.

«««< HEAD ======= Its modular architecture allows different hardware modules and software functionalities to be integrated according to application requirements. This flexibility makes the IOBOX suitable for a wide range of industrial monitoring, automation, and data acquisition applications. »»»> a15e0a982f2f6d5766b8147a7f12c9c22426e956

The system is built with a modular design, meaning it can include various hardware parts and software features based on what's needed for each project. This adaptability makes it useful for many industrial tasks like monitoring, automating, and collecting data.

All these features—like strong embedded processing, multiple communication options, the ability to read both analog and digital signals, and reliable storage—are combined into one easy-to-use device.

2.4 IOBOX Hardware architecture

The microcontroller operates from a 3.3 V power supply and can be programmed and debugged through dedicated JTAG/SWD and UART interfaces. It provides several input and output resources, including four analog input channels connected to the ADC peripherals, seven digital input channels, seven digital output channels, and four PWM outputs for actuator control applications.

For analog signal generation, the platform integrates DAC outputs capable of producing industrial-standard 4–20 mA current signals. These outputs enable the IOBOX to interface directly with industrial control equipment and monitoring systems.

The platform supports a wide range of communication protocols to ensure compatibility with industrial and IoT environments. A USB interface is available for configuration and data exchange, while an SPI interface provides high-speed communication. Multiple I^2C buses are used for sensor integration and peripheral expansion.

Industrial communication capabilities are implemented through dedicated transceivers. To ensure compatibility with modern industrial networks, the IOBOX

integrates a TCAN330GD transceiver that enables CAN FD communication between the STM32G474 and external equipment. LIN communication is achieved through the TJA1020 LIN transceiver, while RS485 communication is implemented using the THVD1410 transceiver, enabling reliable long-distance communication and Modbus RTU integration.

To extend the number of available communication channels, the architecture incorporates a TCP9548A I^2C multiplexer, which expands a single I^2C bus into eight independent channels. Additionally, a PCF8574 I/O expander controls two CD74HC4051 multiplexers, allowing the selection of up to eight UART communication channels through a reduced number of microcontroller pins.

All interfaces are routed to dedicated external connectors, facilitating the connection of sensors, actuators, and external industrial equipment. This modular and scalable architecture provides the flexibility required for industrial automation, monitoring, data acquisition, and IoT applications while maintaining the possibility of future hardware and software extensions.

Figure 2.1 presents the overall synoptic architecture of the IOBOX platform. The system is centered around the STM32G474MBTx microcontroller, which acts as the main processing unit and coordinates data acquisition, signal processing, communication, and control functions.

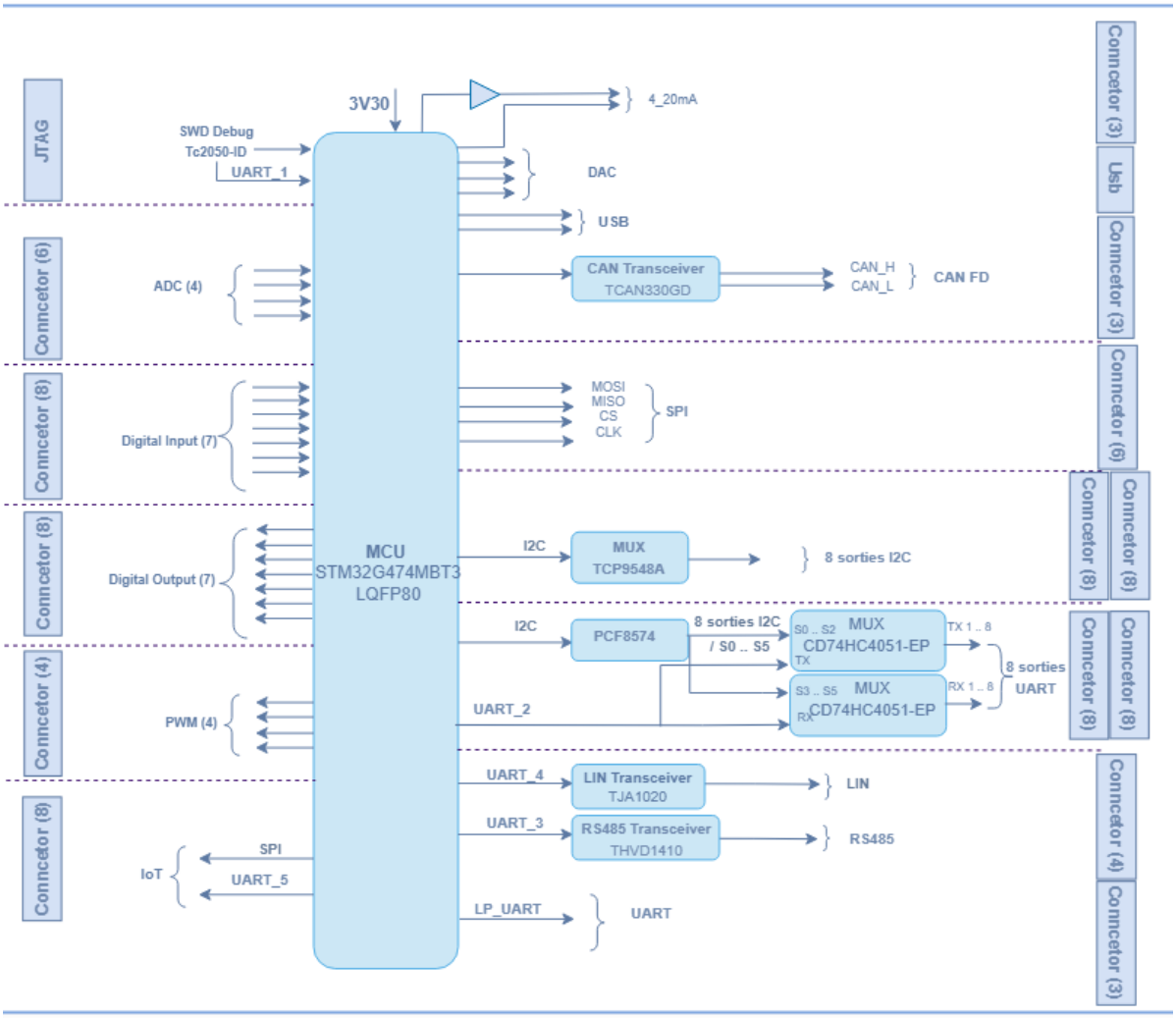

Figure 2.1: Synoptic architecture of the IOBOX platform

2.5 Hardware components

2.5.1 STM32G474MBTx Microcontroller

At the center of the IO-Box hardware design is the STM32G474 microcontroller, a powerful chip made by STMicroelectronics. It uses the ARM Cortex-M4 processor with a floating-point unit, which makes it very good option for handling real-time signal processing and control tasks.

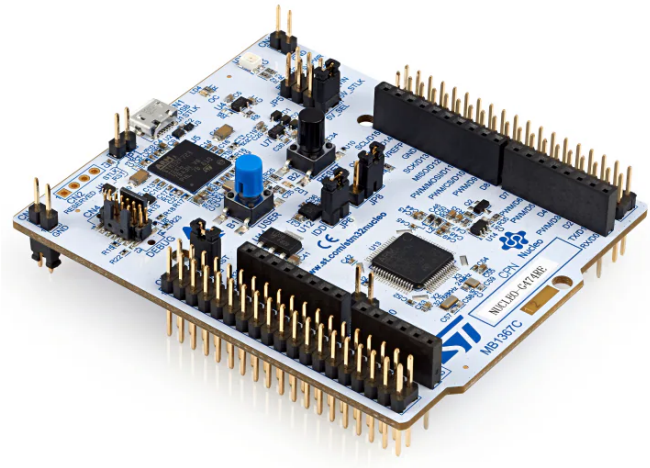

Figure 2.2: STM32G474 Microcontroller (physical package)

The STM32G474 was chosen instead of the STM32F4 series because it has more ADC units (five compared to two on the F4), includes a built-in FDCAN feature, and uses a newer HAL environment. When compared to the STM32L4 series, the G474 has a faster clock speed (170 MHz versus 80 MHz) and a better analog front end, which are important for handling real-time signal processing at the needed sampling rates.

Table 2.1: Product Attributes — STM32G474 Microcontroller

Category	Integrated Circuits, Embedded Systems, Microcontrollers
Manufacturer	STMicroelectronics
Series	STM32G4
Processor Core	ARM Cortex-M4F
Core Size	32-Bit
Speed	170 MHz
Number of I/O	52
Program Memory Size	512 KB Flash
RAM Size	128 KB
Data Converters	ADC: 26 × 12-bit; DAC: 7 × 12-bit
Supply Voltage	1.71 V – 3.6 V
Oscillator Type	Internal
Operating Temperature	–40 °C to +125 °C
Mounting Type	Surface Mount
Package	64-LQFP (10 × 10 mm)
Base Product Number	STM32G474

««««< HEAD Inside the IO-Box, the STM32G474 serves as the main processor. It handles tasks like collecting signals, processing data locally, and controlling actuators. Additionally, it manages communication with other systems using various industrial protocols, which is essential for achieving the project's goals.

2.5.2 EEPROM Memory: M95M01-RDW6TP

To manage the work with the STM32 microcontroller and make sure data is stored reliably, the IO-Box uses an external EEPROM memory called the M95M01-RDW6TP from STMicroelectronics. This device stores important information like settings, calibration data, and system logs, so the system can keep this data even when the power is turned off. ======= Within the IOBOX, the STM32G474 acts as the central processing unit, coordinating signal acquisition, local data processing, and actuator control.

It also supervises communication with external systems through multiple industrial protocols, making it a key enabler of the project's objectives.

2.5.3 EEPROM Memory: M95M01-RDW6TP

To complement the STM32 microcontroller and ensure reliable data storage, the IOBOX integrates an external EEPROM memory, specifically the M95M01-RDW6TP from STMicroelectronics. This device provides non-volatile storage for configuration parameters, calibration data, and system logs, allowing the platform to retain critical information even after power cycles. »»»> a15e0a982f2f6d5766b8147a7f12c9c22426e956

The M95M01-RDW6TP is a 1-Mbit serial EEPROM that is organized into 131,072 memory locations, each holding 8 bits. It has a fast response time of 25 nanoseconds and connects through the SPI bus, which makes it easy to use with the STM32G474. It comes in an 8-pin SOIC package, making it a small and efficient option for embedded systems. It can handle up to one million write operations and keeps data safely for up to 40 years, making it a dependable choice for industrial uses where reliability is key.

Table 2.2: Key Characteristics of M95M01-RDW6TP EEPROM

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Serial EEPROM
Capacity	1 Mbit (131,072 × 8 bits)
Interface	SPI
Access Time	25 ns
Package	SOIC-8
Endurance	1,000,000 write cycles
Data Retention	40 years
Operating Voltage	1.8 V – 5.5 V

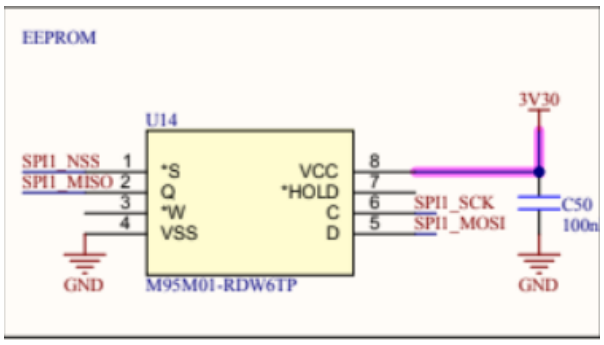

Figure 2.3: M95M01-RDW6TP EEPROM memory chip

2.5.4 Sensors

«««< HEAD In the IOBox project, we added several sensors to measure different environmental and physical conditions. Our main task was to connect these sensors to the STM32G474 microcontroller, set up their settings, handle how they communicate, and do some testing to make sure the data they collect is correct. We looked closely at each sensor, learned about how they work, and wrote down their features to show how each one contributes to the whole system. ======= In the IOBOX project, several sensors were integrated to enable environmental and physical measurements. Our work focused not only on interfacing these sensors with the STM32G474 microcontroller, but also on configuring their registers, managing communication protocols, and performing calibration to ensure accurate data acquisition. Each sensor was studied in detail, and its characteristics were documented to highlight its role within the system. »»»> a15e0a982f2f6d5766b8147a7f12c9c22426e956

ENS210 Sensor

The ENS210 is a digital sensor made by ScioSense that measures both temperature and humidity in a small size. It uses an I^2C interface to connect easily with microcontrollers like the STM32G474. The sensor is known for being accurate and using very little power, which makes it a good choice for use in systems that monitor the environment.

Table 2.3: Key Characteristics of ENS210 Sensor

Feature	**Specification**
Manufacturer	ScioSense
Type	Temperature + Humidity Sensor
Interface	I^2C (16-bit output registers)
Supply Voltage	1.71 V – 3.6 V
Temperature Range	–40 °C to +100 °C
Temperature Accuracy	±0.15 °C
Humidity Range	0 – 100 % RH
Humidity Accuracy	±2 % RH
Package	4-QFN (2 × 2 mm)

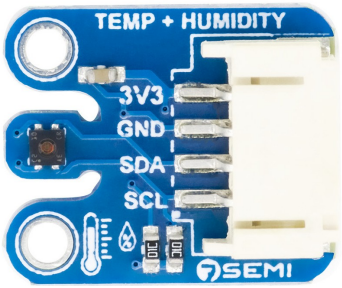

Figure 2.4: ENS210 digital temperature and humidity sensor

OPT3001DNPR Sensor

The OPT3001DNPR is a digital light sensor made by Texas Instruments. It measures how bright the light is and shows the results in units called lux. This makes it great for uses like automatically adjusting screen brightness, controlling display backlights, and monitoring the environment. The sensor uses the I^2C communication method and has built-in calibration and a linear response over a wide range of light levels.

In the IO-Box project, the OPT3001DNPR was used to get precise light readings. Work was done on its internal settings to set up different measurement modes, set thresholds, and run calibration processes, ensuring the sensor gave dependable data for real-time use.

Table 2.4: Key Characteristics of OPT3001DNPR Sensor

Feature	**Specification**
Manufacturer	Texas Instruments
Type	Ambient Light Sensor
Interface	I^2C
Supply Voltage	1.6 V – 3.6 V
Measurement Range	0.01 lux – 83,000 lux
Output Format	16-bit digital result in lux
Accuracy	±10% typical
Operating Temperature	–40 °C to +85 °C
Package	6-Pin WSON (2 × 2 mm)

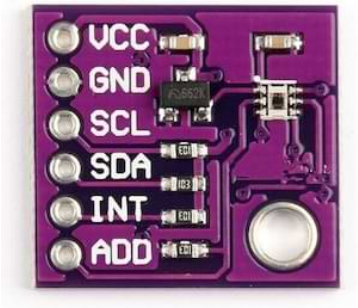

Figure 2.5: OPT3001DNPR ambient light sensor

HTS221TR Sensor

The HTS221TR is a capacitive digital humidity and temperature sensor developed by STMicroelectronics. It integrates sensing elements and signal conditioning circuitry within a compact package, providing calibrated measurements through a standard I^2C interface. The sensor is designed for low-power applications while maintaining good measurement accuracy, making it suitable for environmental monitoring and industrial IoT systems.

Within the IOBOX platform, the HTS221TR is used to acquire ambient temperature and relative humidity data. Its factory-calibrated coefficients simplify integration

and improve measurement reliability. Communication with the STM32G474 microcontroller is achieved through the I^2C bus, and dedicated software drivers were developed to configure the device and process sensor data.

Table 2.5: Key Characteristics of HTS221TR Sensor

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Temperature + Humidity Sensor
Interface	I^2C (SPI mode not used in this project)
Supply Voltage	1.7 V – 3.6 V
Temperature Range	-40 °C to +120 °C
Temperature Accuracy	±0.5 °C (typ.)
Humidity Range	0 – 100 % RH
Humidity Accuracy	±3.5 % RH (typ.)
Output Resolution	16-bit
Package	HLGA-6 (2 × 2 mm)

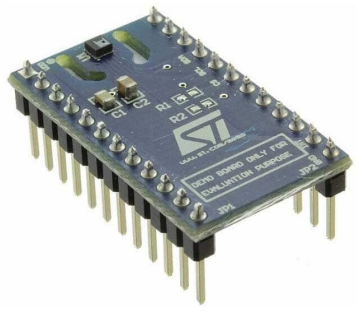

Figure 2.6: HTS221TR temperature and humidity sensor

2.6 Peripherals

2.6.1 Digital-to-Analog Converter (DAC)

The Digital-to-Analog Converter (DAC) is an analog output peripheral integrated within the STM32G474MBTx microcontroller. It converts digital values stored in mem-

ory into continuous analog voltage levels with a resolution defined by the DAC bit width. This allows the generation of precise voltage references or analog waveforms such as sine waves, ramps, or arbitrary signals.

The DAC can operate in several modes, including software-triggered mode, timer-triggered mode, and DMA-driven mode. In timer-triggered mode, a hardware timer periodically updates the DAC output, ensuring precise and deterministic signal timing. When combined with Direct Memory Access (DMA), the DAC can continuously output precomputed waveform data without CPU intervention, significantly improving efficiency.

In the IOBox platform, the DAC is used to generate analog reference signals for actuator control. Two DAC channels are available, supporting both software-triggered and timer-triggered output modes.

An example of DAC signal generation is illustrated in Figure 2.7.

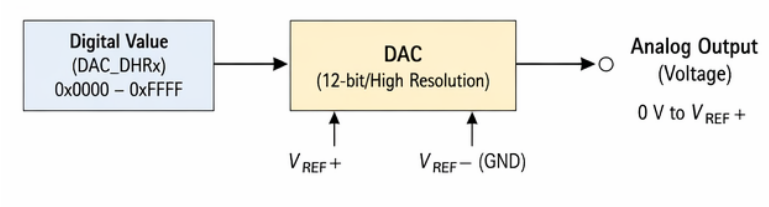

Figure 2.7: Example of analog signal generated by DAC

2.6.2 Analog to Digital Converter (ADC)

The Analog-to-Digital Converter (ADC) is a fundamental peripheral in embedded systems, responsible for transforming continuous analog signals into discrete digital values that can be processed by the microcontroller. This conversion is essential for applications where physical phenomena such as temperature, pressure, or current must be measured and interpreted by digital logic.

In the IOBox project, ADCs are employed to acquire signals from sensors and external modules, enabling precise monitoring and control. The STM32G474 microcontroller integrates multiple high-resolution ADC channels, allowing simultaneous sampling of different inputs. This capability is particularly important in industrial contexts where multiple signals must be captured in real time.

The operation of an ADC follows a well-defined process. The analog input is sampled at regular intervals determined by the sampling frequency. Each sample is then quantized into a digital value according to the converter's resolution (e.g., 12-bit or 16-bit). The resulting digital word represents the amplitude of the analog signal at that

instant. The accuracy of this conversion depends on factors such as resolution, sampling rate, and reference voltage stability.

Key characteristics of ADCs include:

- **Resolution**: Defines the number of discrete levels available.
- **Sampling Rate**: Determines how frequently the analog signal is measured, affecting the fidelity of dynamic signals.
- **Reference Voltage**: Sets the maximum measurable input range, ensuring proper scaling of the digital output.
- **Input Channels**: Multiple channels allow acquisition from several sensors simultaneously.
- **Conversion Modes**: Support for single, continuous, and DMA-driven conversions, enabling flexible integration into firmware.

Within the IO-Box, ADCs are used to capture sensor data such as voltage levels, current measurements, or other analog signals. These values are then processed by the firmware to generate meaningful information, trigger control actions, or communicate results to external supervisory systems.

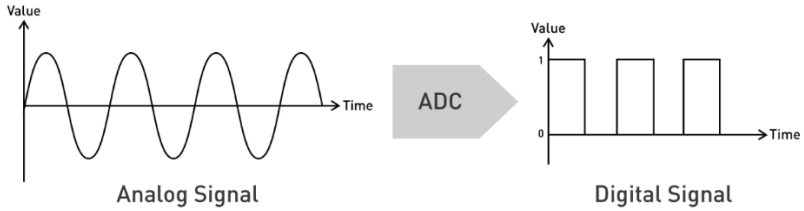

Figure 2.8: Basic ADC conversion from analog input to digital output

2.6.3 Timers

Timers are fundamental peripherals in the STM32G474 microcontroller, designed to provide accurate time base generation, event counting, and signal measurement. They operate by incrementing or decrementing a counter register according to a clock source, and can trigger actions when predefined values are reached.

Key functionalities of timers include:

- **Time Base Generation**: Creating precise delays or periodic events.

- **Input Capture**: Measuring external signal characteristics such as frequency or pulse width.
- **Output Compare**: Generating events when the counter matches a reference value.
- **Synchronization**: Coordinating multiple timers for complex control tasks.
- **Advanced Features**: Dead-time insertion, break inputs, and complementary outputs for safe power electronics control.

Timers are essential for scheduling tasks, measuring sensor signals, and driving peripherals that require precise timing.

2.6.4 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a technique used to encode analog information into a digital waveform by varying the duty cycle of a periodic signal. In embedded systems, PWM is widely used for motor control, LED dimming, and power regulation.

PWM signals in the STM32G474 are generated using timers. The timer counts up to a predefined value stored in the auto-reload register (ARR), while the compare register (CCR) defines the switching point of the output. The ratio between the high time and the total period represents the duty cycle. By adjusting ARR and CCR, both frequency and duty cycle can be controlled with high precision.

Key characteristics of PWM include:

- **Frequency**: Determined by prescaler and ARR configuration.
- **Duty Cycle**: Defined by CCR relative to ARR.
- **Resolution**: Limited by timer bit width (typically 16-bit).
- **Complementary Outputs**: Enable safe control of MOSFET drivers in half-bridge or full-bridge circuits.
- **Applications**: Motor drivers, LED dimming, switching converters, and DAC emulation.

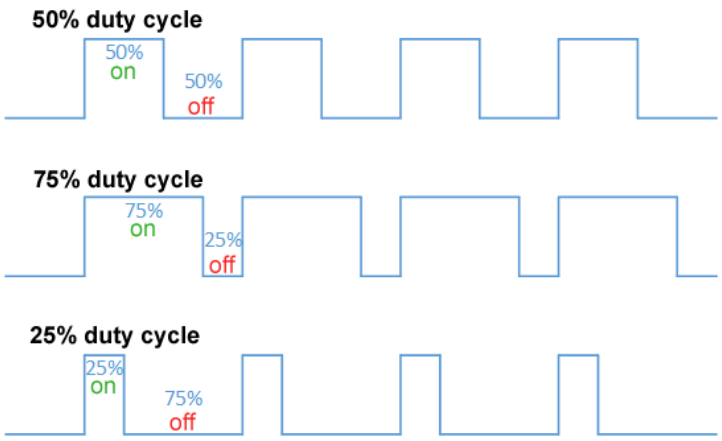

Figure 2.9: Illustration of PWM signals with different duty cycles

2.7 Communication Protocols

2.7.1 I2C Protocol

Inter-Integrated Circuit (I2C) is a synchronous, half-duplex serial communication protocol widely used for communication between integrated circuits over short distances. It relies on two bidirectional lines: the Serial Data Line (SDA) and the Serial Clock Line (SCL), both requiring pull-up resistors. The protocol follows a master-slave architecture, where the master initiates communication and generates the clock signal.

Each device connected to the bus is identified by a unique address. Data is then exchanged one byte at a time, with each byte followed by an acknowledgment bit (ACK/NACK) from the receiving device. The communication is terminated by a STOP condition. I2C supports different speed modes, including Standard Mode (100 kHz), Fast Mode (400 kHz), and higher-speed variants.

The I2C frame structure is illustrated in Figure 2.10.

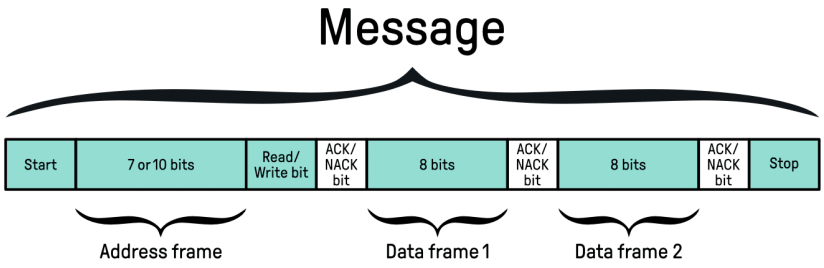

Figure 2.10: I2C frame format

In this project, the I2C interface of the STM32G474MBTx is configured using the

HAL library generated by STM32CubeMX. It is used to communicate with external peripherals such as sensors and low-speed devices. The implementation relies on interrupt or polling-based data transfers depending on timing constraints. Particular attention is given to bus stability, including proper pull-up resistor sizing and handling of potential communication errors such as bus busy states or missing acknowledgments.

As shown in Figure 2.11, the I2C bus allows multiple slave devices.

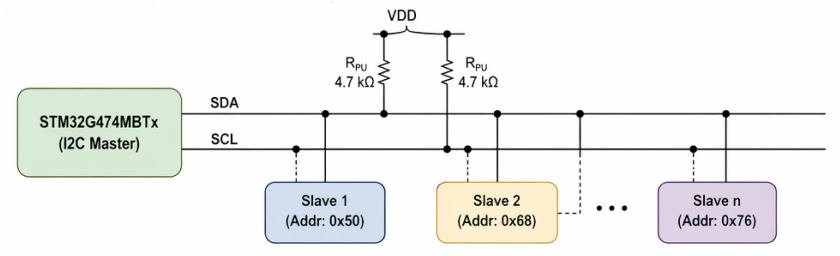

Figure 2.11: I2C communication bus architecture

2.7.2 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used in embedded systems. It enables high-speed data exchange between a microcontroller and peripheral devices such as sensors, memory modules, or communication transceivers. SPI operates in a master–slave configuration, where the microcontroller typically acts as the master, controlling the clock and data flow.

In the IOBOX project, SPI is employed to ensure reliable and efficient communication with external modules that require fast data transfer. Its simplicity and speed make it suitable for real-time applications, particularly when multiple peripherals must be interfaced with the STM32 microcontroller.

SPI communication is based on a clock signal generated by the master device. Each bit of data is shifted out on the MOSI line while simultaneously another bit is shifted in on the MISO line. This process is synchronized by the clock (SCLK), ensuring deterministic timing. The communication sequence typically follows these steps: the master activates the Chip Select (CS) line of the desired slave, provides the clock signal, transmits data bit-by-bit on MOSI while receiving data on MISO, samples the data according to the configured mode, and finally deactivates the CS line to end the session. This mechanism allows full-duplex communication, meaning data can be transmitted and received simultaneously.

SPI supports four modes of operation (Mode 0 to Mode 3), determined by the configuration of clock polarity (CPOL) and clock phase (CPHA). This flexibility ensures

compatibility with a wide range of peripheral devices. Key characteristics of SPI include its high speed (operating at several MHz), simple wiring with four main signals (MOSI, MISO, SCLK, CS), and scalability through multiple slave connections.

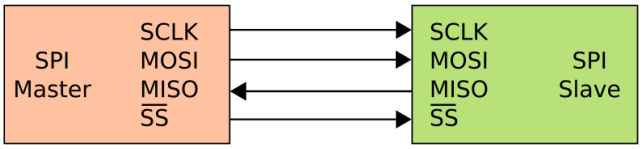

Figure 2.12: Basic SPI communication between master and slave devices

2.7.3 UART Protocol

Universal Asynchronous Receiver-Transmitter (UART) is an asynchronous serial communication protocol that enables full-duplex data exchange between devices without a shared clock signal. Data is transmitted over two lines: Transmit (TX) and Receive (RX). Synchronization between devices is achieved by configuring identical communication parameters, including baud rate, data bits, parity, and stop bits.

Each data frame typically consists of a start bit, followed by a configurable number of data bits (usually 8), an optional parity bit for error detection, and one or more stop bits. Unlike synchronous protocols, UART does not include a clock line, making it simpler but more sensitive to timing mismatches.

In this project, UART is used for communication between the STM32 microcontroller and external systems such as a host computer or other embedded modules. It plays a key role in debugging, logging, and real-time data exchange. The implementation uses the STM32 HAL drivers with support for interrupt-driven and DMA-based transmission to ensure efficient and non-blocking communication. Error handling mechanisms, including overrun, framing, and noise errors, are also considered to improve communication reliability.

The UART frame structure is illustrated in Figure 2.13.

2.7.4 RS485 Communication

RS485 is a robust differential serial communication standard widely used in industrial environments due to its high noise immunity and long-distance capabilities. Unlike single-ended UART communication, RS485 transmits data over a differential pair of lines (A and B), where the information is encoded as a voltage difference between the

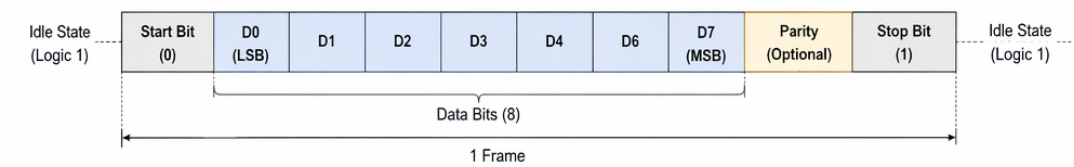

Figure 2.13: UART frame format

two conductors. This approach significantly reduces susceptibility to electromagnetic interference and ground potential differences.

The RS485 standard supports half-duplex communication in multi-drop configurations, allowing multiple nodes to share the same bus. However, only one device can transmit at a time, which requires proper bus arbitration at the software level or through protocol design.

In this project, RS485 serves as the physical layer for Modbus RTU communication. A dedicated UART peripheral (USART3) is used with an external RS485 transceiver to convert TTL logic levels to differential signals. Direction control is managed in software, as described in Chapter 3.

Before transmission, the microcontroller sets the DE pin high to enable the driver stage. After sending the data frame, the DE pin is reset to return the transceiver to reception mode. This timing control is critical to avoid bus contention. The communication is typically used for reliable data exchange in noisy environments, especially when longer cable lengths are required compared to standard UART.

Figure 2.14 illustrates a typical RS485 bus configuration.

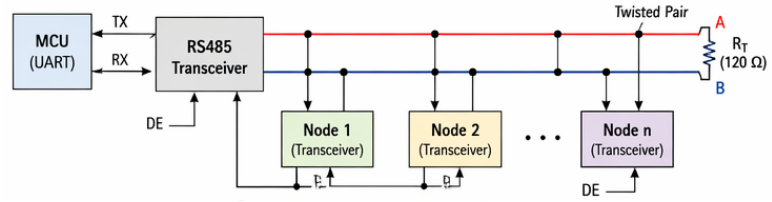

Figure 2.14: RS485 differential bus with multi-node configuration

2.7.5 Modbus Protocol

Modbus is one of the most widely used industrial communication protocols for exchanging data between electronic devices. Originally developed by Modicon in 1979, it follows a client-server (master-slave) architecture and enables communication between programmable logic controllers (PLCs), sensors, actuators, Human-Machine Interfaces

(HMIs), and embedded systems.

Modbus defines a standardized message structure that allows devices from different manufacturers to communicate using a common protocol. Two main variants are commonly used: Modbus RTU, which operates over serial communication interfaces such as RS232 and RS485, and Modbus TCP, which operates over Ethernet networks.

In the IOBOX project, Modbus RTU is implemented over the RS485 physical layer to ensure reliable communication in industrial environments. The STM32G474 microcontroller acts as either a Modbus master or slave depending on the application requirements. Through Modbus registers, external systems can access sensor measurements, configuration parameters, status information, and control commands.

In this project, the IOBox operates as a Modbus RTU slave at address 0x01, communicating over RS485 at 19,200 bps. The slave implementation supports function codes FC01 through FC06, FC15, and FC16. Input registers expose real-time sensor measurements (temperature, humidity, illuminance) to any connected Modbus master, and holding registers allow actuator control. The implementation details are described in Chapter 3.

A Modbus RTU frame consists of four main fields:

- **Slave Address**: Identifies the target device.
- **Function Code**: Specifies the requested operation.
- **Data Field**: Contains transmitted data or register information.
- **CRC Field**: Provides error detection for frame integrity.

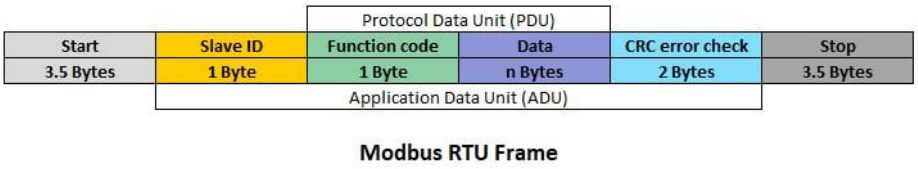

Figure 2.15: Typical Modbus RTU frame structure

To ensure compatibility with standard industrial devices, the implementation supports a subset of Modbus function codes, classified according to their role in data access and control:

Table 2.6: Supported Modbus Function Codes in the IOBOX Platform

Function Code	**Name**	**Description**
FC01	Read Coils	Reads digital output states (ON/OFF status)
FC02	Read Discrete Inputs	Reads digital input signals from sensors or switches
FC03	Read Holding Registers	Reads configuration and control registers (16-bit)
FC04	Read Input Registers	Reads real-time measurement data (sensor values)
FC05	Write Single Coil	Controls a single digital output (ON/OFF command)
FC06	Write Single Register	Writes a single 16-bit configuration or control value
FC15	Write Multiple Coils	Updates multiple digital outputs in one request
FC16	Write Multiple Registers	Writes multiple registers for bulk configuration/control

This classification enables structured interaction with the IOBOX memory map, separating real-time acquisition data (input registers) from controllable parameters (holding registers). It also ensures interoperability with a wide range of Modbus-compatible supervisory systems such as SCADA platforms and PLCs.

2.7.6 Controller Area Network (CAN)

The Controller Area Network (CAN) is a robust serial communication protocol originally developed for automotive applications, but now widely used in industrial and embedded systems. It enables reliable data exchange between multiple devices without requiring a central host, making it ideal for distributed control systems.

In the IOBOX project, CAN is employed to ensure communication with industrial equipment and sensors that demand high reliability and fault tolerance. Its ability to handle multiple nodes on a single bus makes it particularly suitable for environments where numerous devices must share information in real time.

CAN communication is based on a message-oriented protocol. Instead of addressing specific devices directly, each message carries an identifier that defines its priority and content. This allows multiple nodes to listen simultaneously and decide whether to act on the received data. The arbitration mechanism ensures that when two nodes attempt to transmit at the same time, the message with the highest priority (lowest identifier value) wins, while the other node waits until the bus is free. This guarantees deterministic communication without collisions.

Key characteristics of CAN include:

- **Multi-master capability**: Any node can initiate communication when the bus is idle.
- **Error detection and handling**: Built-in mechanisms such as CRC checks, acknowledgment bits, and error frames ensure data integrity.
- **Priority-based arbitration**: Messages are prioritized by identifiers, ensuring that critical data is transmitted first.
- **Scalability**: Supports up to 112 nodes on a single bus, making it suitable for complex systems.
- **Speed**: Operates up to 1 Mbps in classical CAN, and higher in CAN FD (Flexible Data-Rate), which extends payload size and transmission speed.

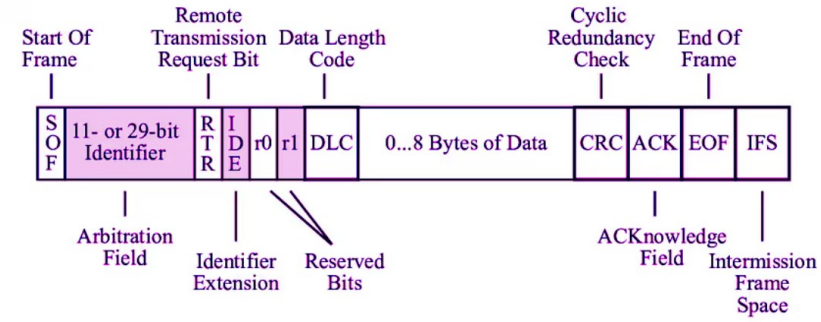

Figure 2.16: Basic CAN frame

In practice, CAN uses two differential lines, CAN_H and CAN_L, to transmit data. This differential signaling improves noise immunity and allows reliable communication even in electrically harsh environments. Each frame consists of fields such as the start of frame, identifier, control bits, data payload, CRC, and end of frame, ensuring structured and error-resistant communication.

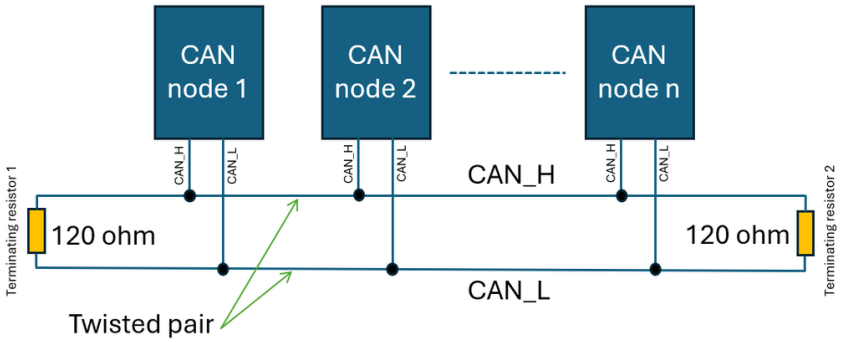

Figure 2.17: Basic CAN bus communication with multiple nodes

2.7.7 LIN Protocol

Local Interconnect Network (LIN) is a low-cost serial communication protocol designed for distributed control systems, particularly in automotive applications. It operates on a single-wire physical layer and follows a strict master-slave architecture where the master node controls the entire bus communication schedule.

Unlike event-driven protocols, LIN is time-triggered, meaning communication is organized into predefined frames sent at fixed intervals. Each frame consists of a header transmitted by the master and a response transmitted by a slave node. The header includes a break field (used to signal the start of a frame), a synchronization byte, and an identifier field that determines which node should respond.

The data response typically contains up to 8 bytes, followed by a checksum used for error detection. LIN achieves deterministic timing by enforcing strict scheduling, making it suitable for non-critical but predictable control tasks.

In this project, LIN communication is implemented using the STM32 UART peripheral configured in LIN mode. The hardware support simplifies break detection and synchronization field handling, reducing software complexity. The system can therefore reliably detect frame boundaries and process incoming data with minimal CPU overhead.

The LIN frame structure is shown in Figure 2.18.

2.8 Software tools

2.8.1 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor optimized for modern software development. It supports multiple programming languages

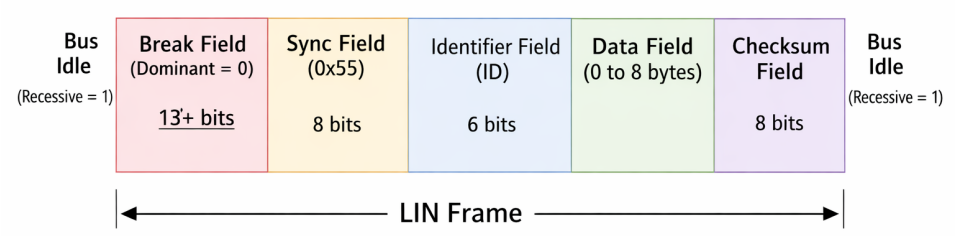

Figure 2.18: LIN frame structure

and provides advanced features such as syntax highlighting, intelligent code completion, and debugging tools.

In this project, Visual Studio Code was used as the primary text editor for documentation, script editing, and Git-based version control. Extensions such as Git Graph enabled visualization of commit history, branch management, and tracking of code evolution across the development team.

Figure 2.19: Visual Studio Code interface

2.8.2 IAR Embedded Workbench

IAR Embedded Workbench (IAR EW) is a professional integrated development environment (IDE) dedicated to embedded systems programming, particularly for microcontrollers such as the STM32 family. It provides a highly optimized C/C++ compiler, advanced debugging capabilities, and efficient project management tools.

In this project, IAR EW was used for firmware development, compilation, and debugging. Its powerful optimization features contributed to generating efficient and reliable embedded code, which is critical for real-time applications and resource-constrained systems[1].

Figure 2.20: IAR Embedded Workbench IDE

2.8.3 STM32CubeMX

STM32CubeMX is a graphical configuration tool developed by STMicroelectronics for STM32 microcontrollers. It allows developers to easily configure peripherals, clock settings, and pin assignments through an intuitive interface. The tool also generates initialization code based on the selected configuration, significantly reducing development time and minimizing configuration errors.

In this project, STM32CubeMX was used in a limited capacity: it was invoked once at the beginning of the project to generate the initial project skeleton, configure the system clock (HSI PLL at 170 MHz), and enable the Serial Wire Debug interface. The peripheral initialization functions that CubeMX would normally generate were intentionally not used. Instead, all peripheral drivers were developed as handwritten BSP (Board Support Package) modules, which are described in detail in Chapter 3. CubeMX thus served exclusively as a project scaffolding and clock configuration tool.

Figure 2.21: STM32CubeMX configuration interface

2.8.4 GitLab

GitLab was used as the version control and collaboration platform throughout the project. The repository was organized by module (BSP, Managers, Application), with branches used to isolate feature development and avoid conflicts during parallel work.

Commit history served as a development log, and merge requests were used to review code before integrating changes into the main branch.

Figure 2.22: GitLab logo

2.8.5 Conclusion

In this chapter, we presented the general concept and system components of the IO-Box project. We began with the synoptic diagram, which outlined the main functional phases of acquisition, filtering, regulation, and transmission/actuation. We then introduced the software architecture, organized into four distinct layers (HAL, BSP, Manager, and Application), and complemented this with the hardware architecture, highlighting the STM32G474 microcontroller, power supply design, communication interfaces, and peripheral modules. Finally, we described the development tools and protocols that support the implementation of the system.

By establishing this technical foundation, Chapter 3 has clarified the resources and methodologies that will be employed in the realization phase. The next chapter will be dedicated to the practical implementation of the IO-Box, detailing the firmware design, register configuration, BSP development, and integration of sensors and actuators into a coherent and functional platform.

CHAPTER 3

DESIGN AND PRACTICAL IMPLEMENTATION

3.1 Introduction

The preceding chapters established the project context, defined the platform requirements, and presented the hardware components and communication protocols that form the technical foundation of the IOBOX platform. This chapter details the design decisions and practical implementation work carried out to bring that foundation into a functional embedded system.

The implementation is organized around a layered software architecture that separates hardware-specific code from higher-level application logic. At the lowest level, a Board Support Package (BSP) provides direct peripheral drivers for every hardware interface present on the platform. Above it, a Manager layer encapsulates the communication protocols and sensor drivers, exposing clean, hardware-independent APIs to the rest of the system. At the top, the Application layer coordinates all modules through a lightweight cooperative task scheduler and assembles sensor data into structured output frames.

The chapter begins by describing the hardware architecture of the IOBOX platform, followed by a detailed account of the software architecture and its constituent layers. Each subsection covers the design rationale, the implementation approach, and the key technical choices made during development. The chapter concludes with a summary of the results obtained and a discussion of the platform's readiness for industrial deployment.

3.2 Hardware architecture

3.3 Software Architecture

The firmware of the IOBOX platform is organised into three distinct horizontal layers: the Board Support Package (BSP), the Manager layer, and the Application layer. This separation was adopted to isolate hardware-specific code from protocol logic and application behaviour, enabling each layer to evolve independently. A central configuration file, `feature_config.h`, governs the compile-time inclusion of every hardware feature through preprocessor flags, ensuring that only the peripherals and protocols required for a given deployment are compiled into the firmware binary.

A single global structure, `SystemBoard`, declared in `feature_config.h` and instantiated in `main.c`, serves as the shared data exchange point between all layers. It aggregates real-time sensor readings, CAN frame data, and status information, and is accessed directly by the Manager and Application layers during each execution cycle.

Figure 3.1: Layered software architecture of the IOBOX platform

The following subsections describe each layer in detail, from the lowest hardware abstraction level up to the application logic.

3.3.1 Board Support Package Layer

The Board Support Package constitutes the lowest layer of the firmware stack. It provides a collection of peripheral driver modules, each encapsulating the initialisation, configuration, and data transfer routines for a single hardware interface. By confining all register-level and HAL-level operations to this layer, the upper layers are shielded from hardware-specific details and can interact with peripherals exclusively through the BSP's public API.

Each BSP module follows a consistent structure: a header file located under `BSP/Inc/` declares the public API, while the corresponding source file under `BSP/Src/` contains the implementation. The platform exposes BSP modules for the following peripheral categories.

Serial communication interfaces are covered by `BSP_USART1`, `BSP_USART2`, `BSP_UART5`, and `BSP_LPUART1` for general-purpose asynchronous communication, by `BSP_RS485` for the half-duplex differential bus used by the Modbus RTU stack, by `BSP_LIN` for the single-wire LIN bus, and by `BSP_UART_MUX` for the software-controlled UART channel multiplexer. High-speed synchronous communication is handled by three independent

SPI drivers: `BSP_SPI1`, `BSP_SPI2`, and `BSP_SPI4`. Sensor and peripheral expansion buses are served by `BSP_I2C2` and `BSP_I2C3`, complemented by `BSP_I2C_MUX`, which controls the TCA9548A eight-channel I^2C multiplexer. The CAN interface is abstracted by `CAN1_BSP`, and digital input/output operations are centralised in `BSP_DigitalIO`.

Analog peripherals are equally covered: four ADC drivers (`ADC1_BSP`, `ADC3_BSP`, `ADC4_BSP`, `ADC5_BSP`) expose acquisition functions for each active conversion channel, while two DAC drivers (`BSP_DAC1` and `BSP_DAC2`) provide voltage output generation. Four timer drivers (`TIM1_BSP` through `TIM4_BSP`) manage time-base and PWM generation. A low-power management module, `BSP_LowPower`, groups the power mode transition routines.

This modular decomposition means that porting the firmware to a different STM32 variant requires changes only within the BSP layer, leaving the Manager and Application layers unmodified.

3.3.2 Manager Layer

The Manager layer sits above the BSP and is responsible for implementing device protocols, sensor acquisition sequences, and data processing logic. Each manager module depends exclusively on BSP APIs and writes its results into the shared `SystemBoard` structure, making the layer's outputs immediately available to the Application layer without direct coupling between managers.

Sensor Managers

Three sensor manager modules were developed, one per physical sensor, each handling I^2C multiplexer channel selection, register-level communication, and raw-to-physical conversion.

The HTS221TR manager, implemented in `HTS221TR_Manager.c`, drives the STMicroelectronics capacitive humidity and temperature sensor connected to I^2C multiplexer channel 0. During initialisation, the module reads the sensor's factory calibration coefficients stored in its one-time programmable memory across registers `0x30`–`0x3F`, computes the four reference points for humidity (`H0_rH`, `H1_rH`, `H0_T0_out`, `H1_T0_out`) and the two ten-bit temperature reference points (`T0_degC`, `T1_degC`, `T0_out`, `T1_out`), and stores them in a private `HTS221_Calibration_t` structure. Subsequent acquisitions apply linear interpolation between these reference points to convert the 16-bit raw ADC output into calibrated temperature in degrees Celsius and relative humidity in percent, as shown in Equation 3.1 and Equation 3.2.

$$\%\mathrm{RH} = H_{0\_rH} + \frac{(H_{1\_rH} - H_{0\_rH}) \cdot (\mathrm{raw} - H_{0\_T0\_out})}{H_{1\_T0\_out} - H_{0\_T0\_out}} \tag{3.1}$$

$$T\ [^{\circ}\mathrm{C}] = T_{0\_degC} + \frac{(T_{1\_degC} - T_{0\_degC}) \cdot (\mathrm{raw} - T_{0\_out})}{T_{1\_out} - T_{0\_out}} \tag{3.2}$$

Data readiness is verified by polling the `STATUS_REG` register before each acquisition. If the data-ready flag is not asserted within a configurable retry count, the function returns `HTS221_TIMEOUT` without blocking the system further.

The ENS210 manager, implemented in `ENS210_manager.c`, interfaces with the ScioSense combined temperature and humidity sensor on multiplexer channel 2. The module operates in single-shot mode: a measurement is triggered by writing `0x03` to the `SENS_START` register, a fixed 10-millisecond settling delay is applied, and the 16-bit raw temperature and humidity registers are then read. Conversion follows the datasheet-specified formulae: temperature is obtained as $T_K = \mathrm{raw}/64$, then converted to Celsius by subtracting 273.15; relative humidity is obtained as $\%\mathrm{RH} = \mathrm{raw}/512$.

The OPT3001DNPR manager, implemented in `OPT3001DNPR_manager.c`, drives the Texas Instruments ambient light sensor on multiplexer channel 1. The sensor is configured at initialisation by writing the word `0xC410` to its `CONFIG` register, selecting continuous conversion mode, automatic full-scale range, and an 800-millisecond conversion time. The 16-bit result register encodes the measurement as a four-bit binary exponent and a twelve-bit mantissa; the illuminance in lux is recovered by the expression $E_{lux} = \mathrm{mantissa} \times 0.01 \times 2^{\mathrm{exponent}}$, implemented in `OPT3001_ConvertToLux()`.

In all three managers, the I^2C multiplexer channel is selected via `BSP_I2C_MUX_SelectChannel()` at the start of each transaction and disabled via `BSP_I2C_MUX_DisableAll()` upon completion, preventing bus contention between sensors sharing the same physical I^2C bus.

Modbus RTU Manager

The Modbus RTU stack is implemented in `Modbus_Manager.c` and provides both master and slave roles over the RS485 physical layer abstracted by `BSP_RS485`. The implementation is self-contained: it includes a pre-computed 256-entry CRC-16/IBM lookup table split into high-byte (`crc_table_hi[]`) and low-byte (`crc_table_lo[]`) arrays, a frame builder (`Modbus_BuildFrame()`), a transaction engine (`Modbus_Transaction()`), and a response validator (`Modbus_ValidateResponse()`).

The slave role, which is active in the current platform configuration, operates through interrupt-driven reception. After initialisation, `BSP_RS485_ReceiveToIdle_IT()` arms the UART receiver; when a UART idle condition is detected, the registered callback

`Modbus_Slave_RxEventCallback()` sets the `frame_ready` flag and records the received byte count in the slave handle. The polling function `Modbus_Slave_Poll()`, called periodically from the application scheduler, inspects this flag, validates the CRC and slave address, and dispatches the frame to a dedicated handler according to the function code. Eight function codes are supported: FC01, FC02, FC03, FC04, FC05, FC06, FC15, and FC16. Frames addressed to other slaves are silently discarded; frames with invalid CRC are dropped without response, as required by the Modbus RTU specification. Following each handled frame, the receiver is re-armed immediately to avoid missing subsequent requests.

CAN Manager

CAN frame reception is handled by `CAN_manager.c`, which implements a circular FIFO ring buffer of configurable depth (`CAN_QUEUE_SIZE`) to decouple the interrupt-driven reception from the periodic application processing. The `CAN1_Sniff()` function polls the BSP receive interface and pushes each arriving frame into the queue, advancing the head pointer with modular arithmetic and overwriting the oldest entry if the buffer is full. The application layer retrieves frames through `CAN1_HasFrame()` and `CAN1_PopFrame()`, ensuring that burst traffic does not cause frame loss under normal operating conditions.

EEPROM Manager

Non-volatile data persistence is managed by `EEPROM_manager.c`, which interfaces with the M95M01-RDW6TP serial EEPROM via the SPI BSP layer. The manager exposes store and load primitives consumed by the Application layer to persist the `DataFrame` structure across power cycles.

3.3.3 Filter Manager

Signal processing is an inherent requirement of any data acquisition platform operating in an industrial environment, where sensor outputs are subject to electrical noise, thermal drift, and measurement uncertainty. To address this requirement, the IOBOX platform implements a generic, multi-type digital filter library in `Filter_Manager.c` and `Filter_Manager.h`. The library provides six distinct filter algorithms grouped into three categories — low-pass, high-pass, and Kalman — all accessible through a unified polymorphic interface. Every filter is encapsulated within a single `FilterInstance_t` structure, which carries a `FilterType_t` type tag and a tagged union of `LowPassFilter_t`, `HighPassFilter_t`, and `KalmanFilter_t` states. A single public entry point, `Filter_Update()`, inspects the type tag and dispatches execution to the appropriate internal handler, allowing the calling module to apply any filter algorithm through an identical function

signature regardless of the underlying implementation.

The architecture is entirely stack-based: no dynamic memory allocation is performed anywhere in the library. The maximum SMA ring-buffer depth is bounded at compile time by the constant `FILTER_SMA_MAX_WINDOW = 30`, explicitly limiting stack usage to a predictable worst case, which is a critical property for safety-conscious embedded deployments. The following subsections describe each filter in detail.

Exponential Moving Average Low-Pass Filter

Mathematical Principle. The Exponential Moving Average (EMA) is a first-order recursive low-pass filter defined by the difference equation:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1] \tag{3.3}$$

where $x[n]$ is the current raw input sample, $y[n]$ is the filtered output, $y[n-1]$ is the previous filtered value, and $\alpha \in (0, 1]$ is the smoothing factor. The equation is implemented verbatim in `EMA_Update()` within `Filter_Manager.c`, where `prev_output` holds $y[n-1]$ and is updated in place at each call.

Role in the Platform. The EMA is the most computationally efficient filter in the library, requiring only two multiplications, one addition, and one floating-point register per instance. It is well suited to continuous sensor streams such as the temperature and humidity readings produced by the HTS221TR and ENS210 sensors, where high-frequency noise components must be attenuated without introducing significant latency. Its recursive nature means that all past samples contribute to the current output with exponentially decaying weights, making it a true infinite impulse response (IIR) filter despite its minimal state.

Initialisation and Parameters. A filter instance is configured as an EMA low-pass by calling:

```
Filter_LowPass_EMA_Init(FilterInstance_t *filter, float alpha);
```

This function sets `filter->type` to `FILTER_TYPE_LOWPASS`, selects `LPF_METHOD_EMA`, stores the provided `alpha` value, and initialises `prev_output` to `0.0f`. The smoothing factor α governs the trade-off between noise rejection and response speed: values of α close to zero apply heavy smoothing at the cost of increased settling time, while values close to one allow rapid tracking of signal changes with minimal noise attenuation. The filter state can be cleared at any time without altering the configured α by calling `Filter_LowPass_Reset()`.

Advantages and Limitations. The EMA demands constant time $O(1)$ computation and $O(1)$ memory regardless of the degree of smoothing applied, making it particularly attractive on memory-constrained microcontrollers. Its primary limitation is the absence of a linear phase response: the group delay of an EMA is frequency-dependent, which means that different frequency components of the signal experience different delays. For slowly varying environmental measurements, this is not a practical concern, but it becomes relevant if the filtered signal is used in a phase-sensitive control loop.

Simple Moving Average Low-Pass Filter

Mathematical Principle. The Simple Moving Average (SMA) computes the arithmetic mean of the N most recent samples:

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \tag{3.4}$$

where N is the window size. The implementation in `SMA_Update()` avoids recomputing the full sum at every step by maintaining a running sum and a fixed-size circular ring buffer of depth `window_size`. When the buffer is not yet full, the average is taken over the samples collected so far, preventing an initial transient caused by zero-padding. Once the buffer is full, each new sample replaces the oldest entry at position `write_index`, the running sum is updated as sum $+= x[n] - x_{\text{old}}$, and `write_index` is advanced modulo `window_size`. This sliding-window approach reduces the per-step arithmetic to two additions and one division, regardless of the window size.

Role in the Platform. The SMA provides a linear-phase low-pass response, meaning all frequency components of the input are delayed by the same amount $(N-1)/2$ samples). This property makes the SMA preferable to the EMA whenever phase consistency is required, such as when two filtered channels are compared or when the filtered output feeds a derivative-based computation. In the IOBOX context, the SMA is applicable to slowly sampled sensor channels where a window of up to 30 samples can be accumulated without introducing unacceptable latency at the 1000 ms sampling period.

Initialisation and Parameters. The SMA instance is configured by calling:

```
Filter_LowPass_SMA_Init(FilterInstance_t *filter, uint8_t window_size);
```

The `window_size` parameter is clamped to the range [1, `FILTER_SMA_MAX_WINDOW`] by the initialisation function. All fields of the `sma` configuration sub-structure — `sample_buffer[]`, `write_index`, `sample_count`, and `sum` — are set to zero during initialisation, ensuring a deterministic startup state.

Advantages and Limitations. The SMA's linear phase response and straightforward parametrisation make it easy to reason about in a design context. Its stop-band attenuation is, however, limited: the frequency response of an N-point SMA has nulls at integer multiples of f_s/N, but exhibits relatively poor roll-off between those nulls. Memory usage scales linearly with window size, with the `FILTER_SMA_MAX_WINDOW` constraint bounding the worst-case allocation to 30 floating-point values (120 bytes) per instance.

First-Order IIR High-Pass Filter

Mathematical Principle. The first-order IIR high-pass filter implemented in `HP_IIR_Update()` is described by the transfer function:

$$H(z) = \frac{1 - z^{-1}}{1 - \alpha z^{-1}} \tag{3.5}$$

which expands into the time-domain recursion:

$$y[n] = \alpha \cdot \big(y[n-1] + x[n] - x[n-1]\big) \tag{3.6}$$

This formulation is implemented exactly as written in the source, with `prev_input` storing $x[n-1]$ and `prev_output` storing $y[n-1]$, both updated on every call. The parameter α determines the cutoff frequency: as $\alpha \to 1$ the cutoff frequency decreases (more of the low frequencies are suppressed), while as $\alpha \to 0$ the filter approaches a pure differentiator.

Role in the Platform. The IIR high-pass filter is used to remove slow-varying drift and DC offsets from sensor signals, isolating only the dynamic, event-driven components of a measurement. In an industrial platform such as the IOBOX, thermal drift in temperature sensors or baseline drift in light sensors can accumulate over long periods and bias downstream processing. A high-pass filter with a low cutoff frequency eliminates these components while preserving abrupt signal changes caused by real physical events.

Initialisation and Parameters. The IIR high-pass filter is initialised by calling:

```
Filter_HighPass_IIR_Init(FilterInstance_t *filter, float alpha);
```

Both `prev_input` and `prev_output` are set to `0.0f` at initialisation. The reset function `Filter_HighPass_Reset()` clears these two state variables without modifying the configured α.

Advantages and Limitations. This filter requires only three state variables (`alpha`, `prev_input`, `prev_output`), making it the most memory-efficient of the three high-pass implementations. Its recursive nature introduces frequency-dependent phase shift, which is acceptable for applications where only the magnitude of signal changes is of interest. For a linear-phase alternative, the SMA-subtraction method described below is available.

Simple Difference High-Pass Filter

Mathematical Principle. The simplest possible discrete-time high-pass operation is the first-order backward difference:

$$y[n] = x[n] - x[n-1] \tag{3.7}$$

implemented in `HP_Difference_Update()` by storing the previous input in `prev_input` and returning the subtraction result. There are no tunable parameters beyond the initial state.

Role in the Platform. The difference filter acts as an approximation to differentiation in the discrete-time domain. Its primary use case in the IOBOX context is step detection and event detection on slowly varying sensor channels: a sudden change in illuminance or temperature produces a large output spike, while a slowly drifting baseline produces near-zero output. Because the filter requires no configuration parameters, it is particularly convenient for rapid prototyping or for situations where the signal statistics are not yet characterised.

Initialisation and Parameters. The filter is configured with no tunable parameters:

```
Filter_HighPass_Difference_Init(FilterInstance_t *filter);
```

The single state variable `prev_input` is initialised to `0.0f`, which introduces a single-sample startup transient equal in magnitude to the first input value. This behaviour is consistent with the zero-state assumption applied uniformly across the library.

Advantages and Limitations. The difference filter is computationally trivial and parameter-free, but its frequency response rises monotonically with frequency at 20 dB/decade, meaning it amplifies high-frequency noise as strongly as it attenuates low-frequency drift. In practice, its use is appropriate only for clean or pre-filtered signals or for event-detection applications where noise spikes and step events can be distinguished by magnitude.

SMA-Subtraction High-Pass Filter

Mathematical Principle. The SMA-subtraction high-pass filter derives its output by subtracting the local moving average from the current sample:

$$y[n] = x[n] - \frac{1}{N} \sum_{k=0}^{N-1} x[n-k] \tag{3.8}$$

The moving average is maintained using the same efficient ring-buffer running-sum technique as the SMA low-pass filter, implemented in `HP_SMA_SUB_Update()`. The method can be understood as the output of a complementary filter pair: if the SMA low-pass output is $y_{LP}[n]$, then the SMA-subtraction high-pass output is $x[n] - y_{LP}[n]$. This complementary relationship means that $y_{LP}[n] + y_{HP}[n] = x[n]$ holds exactly, guaranteeing perfect signal decomposition.

Role in the Platform. Unlike the IIR high-pass filter, the SMA-subtraction method inherits the linear phase property of the underlying SMA, making it suitable for use cases where phase consistency between the low-frequency and high-frequency components of a decomposed signal is required. In the IOBOX platform, this filter can be applied to separate the slowly evolving baseline of an ambient light reading from transient illumination events, while ensuring that the detected transient is time-aligned with the original sample stream.

Initialisation and Parameters. The filter is initialised by calling:

```
Filter_HighPass_SMA_Substract_Init(FilterInstance_t *filter,
                                   uint8_t window_size);
```

The `window_size` parameter is subject to the same clamping as the SMA low-pass: values outside [1, `FILTER_SMA_MAX_WINDOW`] are corrected silently. All ring-buffer state is zeroed at initialisation. The reset function `Filter_HighPass_Reset()` clears the buffer, sum, indices, and sample count while preserving the configured window size.

Advantages and Limitations. The main advantage of this filter over the IIR high-pass is its linear phase response and its intuitive parametrisation through a window length in samples. Its limitation is identical to that of the SMA low-pass: memory scales linearly with window size, and the stop-band rejection is limited by the SMA's moderate roll-off characteristics.

1-State Position Kalman Filter

Mathematical Principle. The one-state Kalman filter, implemented in `Kalman_Position_Update()`, models the measured quantity as a constant subject to additive process noise. The filter operates in two alternating steps at each sample instant.

The prediction step propagates the error covariance P forward in time by adding the process noise variance Q:

$$P_{\text{pred}} = P + Q \tag{3.9}$$

The update step computes the Kalman gain K, corrects the state estimate $\hat{x}$ using the innovation (the difference between the new measurement z and the current estimate), and updates the covariance:

$$K = \frac{P_{\text{pred}}}{P_{\text{pred}} + R} \tag{3.10}$$

$$\hat{x} = \hat{x} + K \cdot (z - \hat{x}) \tag{3.11}$$

$$P = (1 - K) \cdot P_{\text{pred}} \tag{3.12}$$

where R is the measurement noise variance. These four equations are implemented directly and sequentially in `Kalman_Position_Update()`, with `error_covariance`, `state_estimate`, `process_variance`, and `measurement_variance` stored in the `position` sub-structure of `KalmanFilter_t`.

Role in the Platform. The position Kalman filter provides optimal minimum-mean-square-error state estimation for a scalar measurement corrupted by Gaussian noise, under the assumption that the true quantity varies slowly compared to the measurement noise. This makes it well suited to temperature and humidity estimation, where physical conditions change slowly and sensor noise is approximately Gaussian. Compared to the EMA, the position Kalman filter offers the advantage that its effective smoothing coefficient K adapts automatically over time: it starts large (trusting new measurements heavily when initial uncertainty is high) and converges toward a steady-state value as confidence in the estimate accumulates.

Initialisation and Parameters. The filter is initialised by calling:

```
Filter_Kalman_Position_Init(FilterInstance_t *filter,
                            float process_var,
                            float measurement_var,
                            float init_state,
                            float init_cov);
```

The parameter `process_var` represents Q, quantifying how much the true physical quantity is expected to vary between samples. The parameter `measurement_var` represents R, quantifying the sensor noise power. The ratio Q/R governs the steady-state behaviour: a small Q/R yields heavy smoothing (the filter trusts its model more than new measurements), while a large Q/R produces fast tracking (the filter trusts new measurements more). The initial state `init_state` and initial covariance `init_cov` set the starting conditions. The state can be re-initialised without changing Q or R via `Filter_Kalman_Position_Reset()`.

Advantages and Limitations. The position Kalman filter is statistically optimal for linear systems with Gaussian noise, and its adaptive gain behaviour eliminates the need for manual tuning of a fixed smoothing coefficient. Its limitation is the constant-position process model: if the measured quantity exhibits a trend or persistent drift rather than random variation around a constant, the filter will lag behind the true value, as the model does not account for a rate of change. The two-state velocity model described below addresses this limitation.

2-State Constant-Velocity Kalman Filter

Mathematical Principle. The two-state Kalman filter, implemented in `Kalman_Velocity_Update()`, extends the position model by adding a velocity state, yielding the state vector $\mathbf{x} = [\text{pos},\ \text{vel}]^T$. The process model assumes constant velocity with additive noise, expressed by the state transition matrix $\mathbf{F}$ and measurement matrix $\mathbf{H}$:

$$\mathbf{F} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{3.13}$$

The full predict–update cycle is applied at each call. In the prediction step:

$$\mathbf{x}_{\text{pred}} = \mathbf{F}\,\mathbf{x}, \quad \mathbf{P}_{\text{pred}} = \mathbf{F}\,\mathbf{P}\,\mathbf{F}^T + \mathbf{Q} \tag{3.14}$$

In the update step, the Kalman gain vector $\mathbf{K}$, the state correction, and the covariance update are:

$$\mathbf{K} = \frac{\mathbf{P}_{\text{pred}}\,\mathbf{H}^T}{\mathbf{H}\,\mathbf{P}_{\text{pred}}\,\mathbf{H}^T + R} \tag{3.15}$$

$$\mathbf{x} = \mathbf{x}_{\text{pred}} + \mathbf{K}\,(z - \mathbf{H}\,\mathbf{x}_{\text{pred}}) \tag{3.16}$$

$$\mathbf{P} = (\mathbf{I} - \mathbf{K}\,\mathbf{H})\,\mathbf{P}_{\text{pred}} \tag{3.17}$$

The implementation expands all matrix operations analytically for the specific 2×2 case, storing the symmetric covariance matrix in the four-element array `covariance[4]` in row-major order. The symmetry condition `covariance[1] = covariance[2]` is enforced explicitly after the prediction step. The filter returns `state[0]`, the filtered position estimate, as its output.

Role in the Platform. The constant-velocity Kalman filter is appropriate for signals that exhibit consistent trends or ramps rather than stationary behaviour. In an industrial monitoring context, this corresponds to physical processes such as a temperature that rises steadily during equipment warm-up, or illuminance that increases progressively at dawn. By explicitly modelling a velocity state, the filter anticipates the trend in its prediction step and thus tracks the signal with lower latency than a position-only filter under the same noise conditions.

Initialisation and Parameters. The filter is initialised by calling:

```
Filter_Kalman_Velocity_Init(FilterInstance_t *filter,
                            float pos_process_noise,
                            float vel_process_noise,
                            float measurement_var,
                            float init_pos, float init_vel,
                            float init_cov_pos, float init_cov_vel);
```

The process noise covariance matrix $\mathbf{Q}$ is initialised as a diagonal matrix with `pos_process_noise` at position Q_{00} and `vel_process_noise` at Q_{11}, with off-diagonal terms set to `0.0f`. The initial covariance matrix $\mathbf{P}$ is similarly initialised as diagonal, reflecting the assumption of independent initial uncertainties in position and velocity. The state vector is set to $[\texttt{init\_pos},\ \texttt{init\_vel}]^T$. The filter state can be reset to new initial conditions via `Filter_Kalman_Velocity_Reset()`, which accepts new initial position, velocity, and the two diagonal covariance terms while preserving the process and measurement noise configuration.

Advantages and Limitations. The two-state model provides superior tracking performance for signals with persistent trends compared to the constant-position model, at the cost of doubling the state dimension and quadrupling the covariance matrix size. The model assumes that the signal velocity itself changes only due to process noise, which means the filter may exhibit transient overshoot when the velocity of the underlying signal changes abruptly. The quality of the filter's performance is strongly dependent on the accuracy of the process noise parameters: if `vel_process_noise` is set too low, the filter will underestimate velocity changes and lag behind abrupt transitions; if set too high, the velocity estimate will track noise rather than trend.

Unified Filter Interface Summary

All six filter algorithms described above are accessed exclusively through the function `Filter_Update(FilterInstance_t *filter, float input)`, which inspects the `type` field of the `FilterInstance_t` structure and dispatches the call to the appropriate internal handler via a `switch` statement. This design decouples the calling code from the filter implementation: a module that processes a sensor channel can switch from an EMA to a Kalman filter by changing only the initialisation call, without modifying the update loop. The complete filter taxonomy is summarised in Table 3.1.

Table 3.1: Summary of filter algorithms implemented in `Filter_Manager.c`

Category	**Method**	**Init Function**	**State Variables**
Low-pass	EMA	`Filter_LowPass_EMA_Init`	2
Low-pass	SMA	`Filter_LowPass_SMA_Init`	$N + 3$
High-pass	IIR	`Filter_HighPass_IIR_Init`	3
High-pass	Difference	`Filter_HighPass_Difference_Init`	1
High-pass	SMA-Subtraction	`Filter_HighPass_SMA_Substract_Init`	$N + 3$
Kalman	Position (1-state)	`Filter_Kalman_Position_Init`	4
Kalman	Velocity (2-state)	`Filter_Kalman_Velocity_Init`	10

The Filter Manager thus provides the IOBOX platform with a complete, self-contained signal processing toolkit that can be applied to any scalar measurement channel. Its design prioritises predictable memory usage, zero dynamic allocation, and a uniform calling interface, all of which are essential properties for reliable embedded firmware.

3.3.4 Application Layer

The Application layer is the topmost layer of the firmware stack. It does not interact with hardware directly but instead coordinates the Manager modules, orchestrates data flow, and assembles the final output frame. It is composed of four modules: the cooperative task scheduler, the Sensor Module, the Modbus Module, and the Frame Builder.

Cooperative Task Scheduler

Rather than employing a real-time operating system, a lightweight cooperative scheduler was implemented directly in `main.c`. The scheduler is based on a statically allocated array of `TaskEntry` structures, each holding a function pointer (`task_fn`), a period in milliseconds (`period_ms`), and a timestamp of the last execution (`last_ms`). The dispatcher `Application_RunTasks()` iterates over the array on every pass of the main loop and invokes each task function whenever the elapsed time since its last execution meets or exceeds its configured period, using `HAL_GetTick()` as the time reference.

Six tasks are registered in the current configuration, as shown in Table 3.2. All task functions are non-blocking by design: they return immediately if no new data is available or if their internal period has not elapsed, preventing any single task from starving the others.

Table 3.2: Registered tasks in the cooperative scheduler

Task Function	**Period (ms)**	**Responsibility**
`SensorModule_Task`	10	Poll all enabled sensors
`CAN_Manager_Run`	10	Drain CAN FIFO and update `sysboard.can`
`Modbus_Module_Poll`	20	Process incoming Modbus RTU frames
`EEPROM_StoreFrame`	1000	Persist current `SystemBoard` to EEPROM
`EEPROM_LoadTask`	1000	Reload last stored frame from EEPROM
`FrameBuilderTask`	1000	Assemble and finalise the output `IoFrame`

Sensor Module

The Sensor Module, implemented in `Sensor_module.c`, provides a unified acquisition interface for all enabled sensors. It maintains one private timestamp variable per sensor and, on each invocation of `SensorModule_Task()`, compares the current tick

against each timestamp to determine whether the sensor's configured sampling period has elapsed. When it has, the corresponding manager acquisition function is called and the result is written directly into the `sysboard` global structure. The per-sensor sampling periods are defined centrally in `feature_config.h`: 1000 ms for the HTS221TR, 1000 ms for the ENS210, and 1000 ms for the OPT3001DNPR. The module is fully driven by compile-time feature flags; sensors disabled at configuration time introduce no runtime overhead.

Modbus Application Module

The Modbus Application Module, implemented in `Modbus_module.c`, forms the bridge between the Modbus RTU stack and the platform data model. It defines a static `object_map[]` table of `Modbus_ObjectMap` entries, each associating a Modbus register address and object type with a pointer to the corresponding field in `sysboard` and an access permission flag. The current register map exposes five read-only input registers: ENS210 humidity at address 0, ENS210 temperature at address 1, HTS221TR humidity at address 2, HTS221TR temperature at address 3, and OPT3001 illuminance at address 4. Floating-point values are scaled by a factor of 100 before transmission, preserving two decimal places within the 16-bit Modbus register format.

Six callback functions (`ReadInput`, `ReadDiscrete`, `ReadHolding`, `WriteHolding`, `ReadCoil`, `WriteCoil`) are registered in the `Modbus_SlaveTypeDef` handle and invoked by the Modbus Manager dispatcher upon receipt of a valid request.

Table 3.3: Modbus input register map

Address	**Mapped Field**	**Physical Quantity**	**Scale**
0	`sysboard.ens210_hum`	Relative humidity (%)	×100
1	`sysboard.ens210_temp`	Temperature (°C)	×100
2	`sysboard.hts221_hum`	Relative humidity (%)	×100
3	`sysboard.hts221_temp`	Temperature (°C)	×100
4	`sysboard.opt3001_lux`	Illuminance (lux)	×100

Frame Builder

The Frame Builder, implemented in `frame.c`, is responsible for assembling the final output structure that the IOBOX platform transmits to external systems. Rather than forwarding raw sensor readings directly, the platform encapsulates all acquired data

within a structured binary frame, the `IoFrame`, whose layout is illustrated in Figure 3.2. This design ensures that any receiving system can unambiguously identify, validate, and decode the payload regardless of which sensors or modules are active in a given deployment.

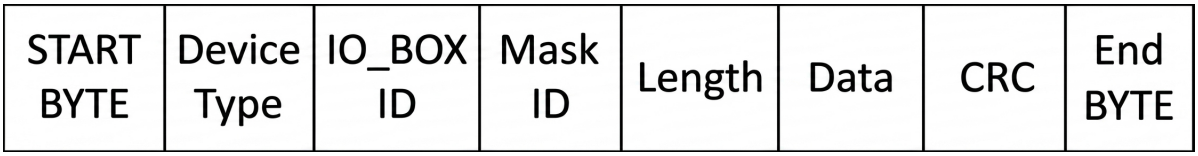

Figure 3.2: Structure of the IOBOX output frame (`IoFrame`)

The `BuildFinalFrame()` function populates each field of the `IoFrame` structure from the `DataFrame` provided by the EEPROM load task, then computes and appends a CRC checksum over all preceding fields. The complete frame format is described in detail below.

Start Byte. The first field of the frame is a fixed synchronisation byte set to the constant value `0xAA`. Its purpose is to signal the beginning of a valid frame to any receiving device scanning an incoming byte stream. By defining a distinctive and consistent delimiter, the receiver can establish frame boundaries without relying on inter-frame timing gaps, which are difficult to guarantee reliably in shared or noisy industrial buses. The value `0xAA` (binary `10101010`) was selected for its alternating bit pattern, which is easily detectable even in the presence of bit-level noise.

Device Type. The Device Type field identifies the category of the transmitting device. In the current platform, this field is set to the compile-time constant `DEVICE_TYPE` defined in `frame.h`. Its presence allows a network to host multiple types of embedded nodes simultaneously: a receiving gateway can inspect this field and apply the appropriate decoding logic without prior knowledge of the sender's identity. For the IOBOX platform, this field unambiguously indicates that the frame originates from an industrial IO acquisition node rather than any other device type on the network.

IOBOX ID. The IOBOX ID field carries the unique numeric identifier of the specific IOBOX unit that generated the frame, populated from the compile-time constant `IOBOX_ID`. This field is essential in multi-node deployments where several IOBOX units share a common communication channel. Without a per-unit identifier, a receiving system would be unable to associate incoming frames with their physical origin, making it impossible to distinguish measurements from different field locations. By embedding the

identifier directly in each frame, the architecture remains stateless at the receiver side: no session or connection establishment is required to route incoming data correctly.

Mask ID. The Mask ID is a configuration bitmap that describes which sensors and communication modules are active and contributing data to the current frame. Each bit of this field corresponds to one module, as detailed in Table 3.4. A bit set to logic one indicates that the associated module is enabled and that its data is present in the payload; a bit set to logic zero indicates that the module is disabled and that its data is absent from the frame.

Table 3.4: Mask ID bit field definitions

Bit	Module	Logic 1 meaning
5	CAN sniffer	CAN data present in payload
4	HTS221TR Temperature	HTS221 temperature present
3	HTS221TR Humidity	HTS221 humidity present
2	ENS210 Temperature	ENS210 temperature present
1	ENS210 Humidity	ENS210 humidity present
0	OPT3001DNPR Illuminance	OPT3001 lux value present

As an illustrative example, a Mask ID value of `0b100110` indicates that the CAN sniffer, ENS210 temperature, and ENS210 humidity modules are active, while the HTS221TR and OPT3001DNPR channels are disabled. The receiver reads this field before parsing the Data field, and uses the active bits to determine the exact byte layout of the payload. This mechanism provides the frame with both flexibility and forward compatibility: new modules can be introduced by assigning previously unused bits to them, without altering the structure of frames generated by existing firmware versions that leave those bits at zero.

Length. The Length field specifies the total size of the Data field in bytes. Its role is to provide the receiver with an explicit boundary for the payload, independently of the Mask ID. Without this field, a receiver would be required to re-derive the payload length from the Mask ID by summing the sizes of all active modules, which introduces coupling between the parser and the bit definitions. By encoding the length directly, the frame becomes self-describing: a receiver can locate the CRC field and the End Byte at a fixed offset from the start of the Data field, even if the set of supported modules differs between firmware versions.

Data. The Data field contains the actual measurement values and communication data collected by the platform during the current acquisition cycle. Its contents are determined entirely by the active bits in the Mask ID field. In the current platform configuration, with all three sensors and the CAN sniffer enabled, the Data field contains: the ENS210 temperature, the ENS210 relative humidity, the HTS221TR temperature, the HTS221TR relative humidity, the OPT3001DNPR illuminance in lux, and the CAN frame payload comprising the message identifier, the RTR flag, the data length code, and up to eight data bytes. All floating-point sensor values are stored as `float` fields within the `IoFrame` structure, preserving full measurement precision within the frame. The architecture is explicitly designed to accommodate future modules: additional sensor types or communication interfaces can be incorporated by assigning new Mask ID bits and appending the corresponding data fields to the payload definition, without disrupting the parsing of frames that do not set those bits.

CRC. The CRC field contains a 16-bit Cyclic Redundancy Check computed over all preceding bytes of the frame, from the Start Byte through the last byte of the Data field. The algorithm implemented in `calc_crc16()` uses the CRC-16/IBM polynomial `0xA001` with an initial value of `0xFFFF`, applied bit-by-bit without a look-up table. The checksum is appended as the second-to-last field of the structure, immediately before the End Byte, and covers exactly `sizeof(IoFrame) - sizeof(finalFrame.crc)` bytes as expressed in the implementation. This placement allows the receiver to verify frame integrity before consuming any measurement data. A CRC mismatch at the receiver indicates either a transmission error or a corrupted EEPROM frame, and the frame must be discarded. The use of CRC-16 over a simple checksum was motivated by its superior error detection capability, particularly its ability to detect all single-bit errors, all double-bit errors, and all burst errors shorter than 16 bits.

End Byte. The frame is terminated by a fixed End Byte set to the constant value `0x55`. Together with the Start Byte `0xAA`, this field constitutes a second layer of frame boundary validation: a receiver that has successfully located a Start Byte, parsed the declared Length, and verified the CRC can use the End Byte as a final consistency check. A missing or incorrect End Byte at the expected position indicates a framing error, for example caused by a length field corruption that displaced the parser's position within the stream. The combination of `0xAA` at the start and `0x55` at the end is also visually symmetric (binary `10101010` and `01010101`), which simplifies manual inspection during debugging.

The complete field layout of the `IoFrame` structure is summarised in Table 3.5.

Table 3.5: IoFrame field summary

Field	**Value / Type**	**Description**
Start Byte	`0xAA` (`uint8_t`)	Frame synchronisation delimiter
Device Type	`DEVICE_TYPE` (`uint8_t`)	Node category identifier
IOBOX ID	`IOBOX_ID` (`uint8_t`)	Per-unit unique identifier
Mask ID	`BUILD_MASK_ID` (`uint8_t`)	Active module bitmap
Length	Payload size (`uint16_t`)	Data field length in bytes
Data	Sensor + CAN values (`float`, `uint8_t[]`)	Active measurement payload
CRC	CRC-16/IBM (`uint16_t`)	Integrity check over all preceding fields
End Byte	`0x55` (`uint8_t`)	Frame termination delimiter

The `FrameBuilderTask()` function retrieves the last EEPROM-loaded `DataFrame` via `EEPROM_GetLastLoaded()` and delegates assembly to `BuildFinalFrame()`, which fills every field sequentially before computing and inserting the CRC. This design ensures that the output frame always reflects the most recently persisted system state, providing continuity across power cycles and making the frame contents auditable through the non-volatile storage layer.

System Data Flow

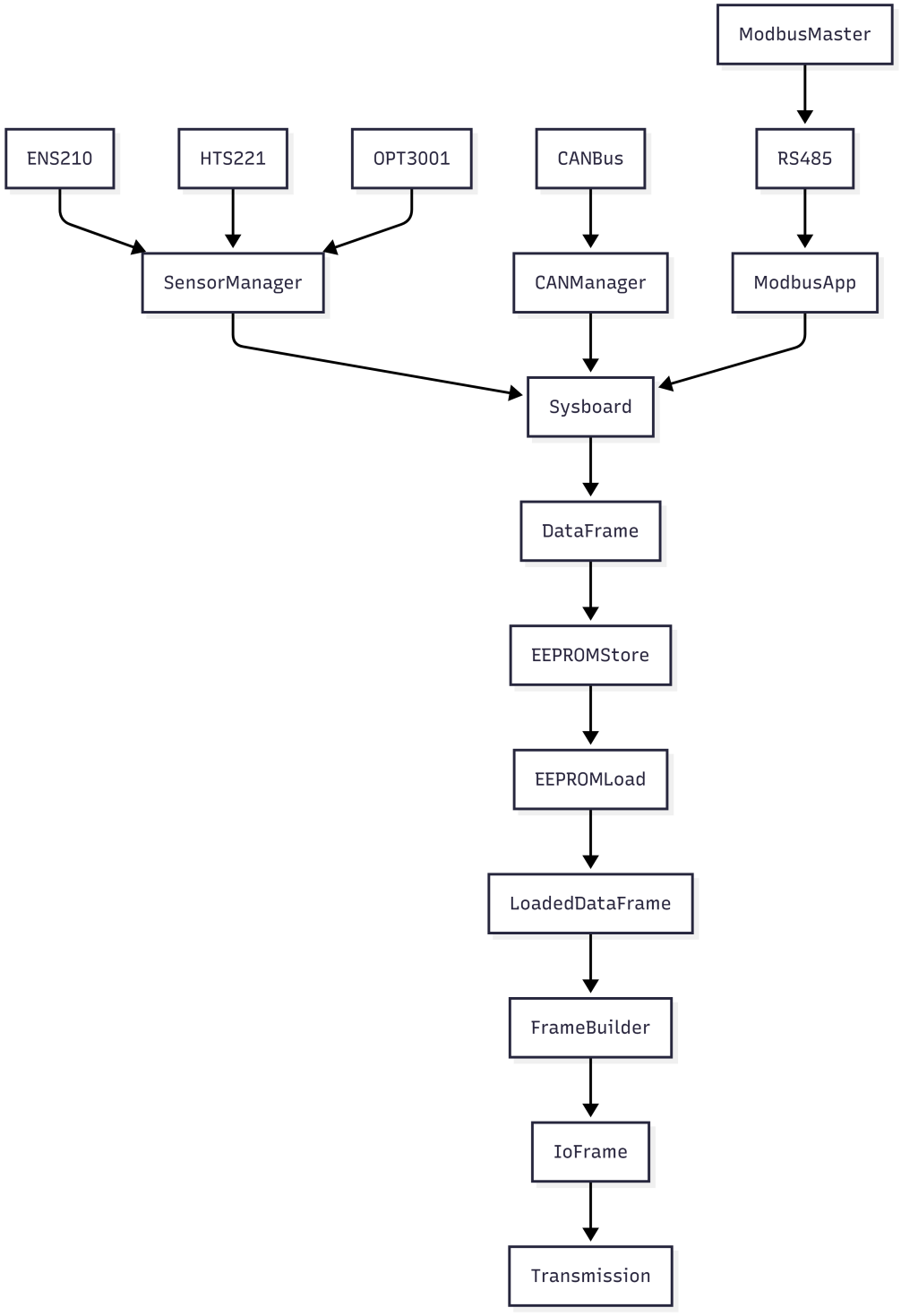

Figure 3.3: End-to-end data flow across the three firmware layers

The complete data path through the system can be summarised as follows. Physical sensor measurements are acquired by the Sensor Module and written into `sysboard`. Simultaneously, CAN frames captured by the CAN Manager are stored in the same structure. At every 1000 ms interval, the EEPROM Store task serialises `sysboard` into a `DataFrame` and writes it to non-volatile memory; the EEPROM Load task immediately retrieves the last valid frame. Finally, the Frame Builder consumes the loaded `DataFrame` and produces the `IoFrame` output, ready for transmission. Throughout this pipeline, any Modbus master connected via RS485 may read live sensor values directly from `sysboard` through the Modbus Application Module at any point in the cycle.

The software architecture described in this section provides a clean separation of concerns, compile-time configurability, and a deterministic execution model without the overhead of a real-time operating system. The following section presents the validation results obtained on the target hardware.

3.4 Conclusion

BIBLIOGRAPHY

[1] IAR EW. consulted on 04/13/2026. https://www.iar.com/.

[2] ISIMM. consulted on 04/13/2026. http://www.isimm.rnu.tn/public/.